

Chapter 4: Serial Communication

4.1 SPI

4.2 I²C

4.3 UART

4.4 USB

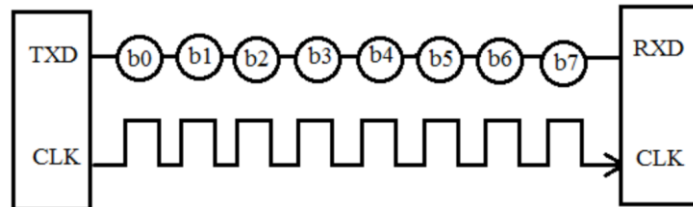
Introduction

❑ Parallel communication

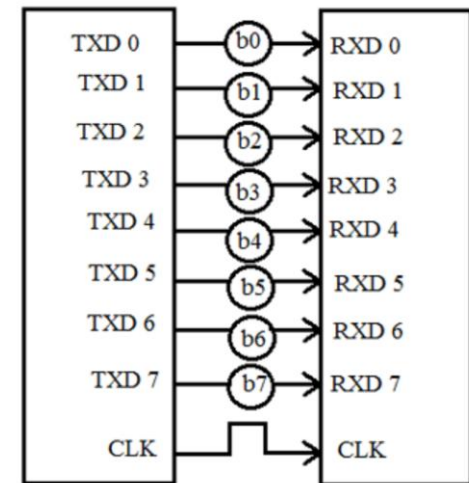
- | Multiple bits are sent and received simultaneously. Multiple GPIO pins are required.
- | Separate signals for data, address, and control. E.g. LCD 1602.

❑ Serial communication

- | Data is sent as a stream of bits, one bit is sent/receive at a time.
- | Require mechanism to synchronize transmitter/receiver bit time.
- | Require very few GPIO pins.



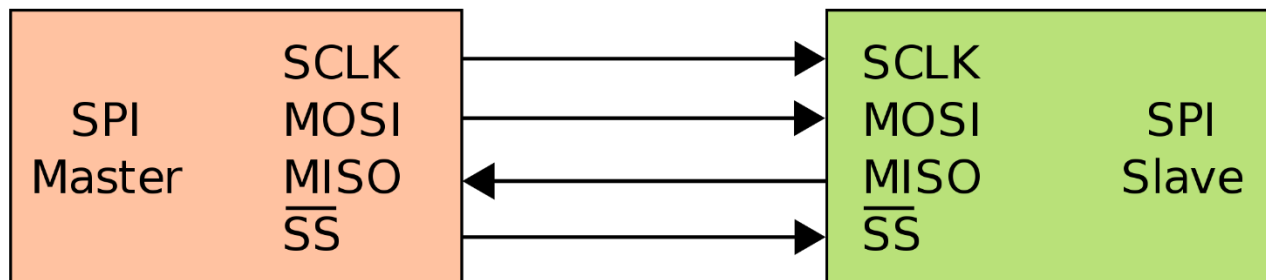
Serial communication



Parallel communication

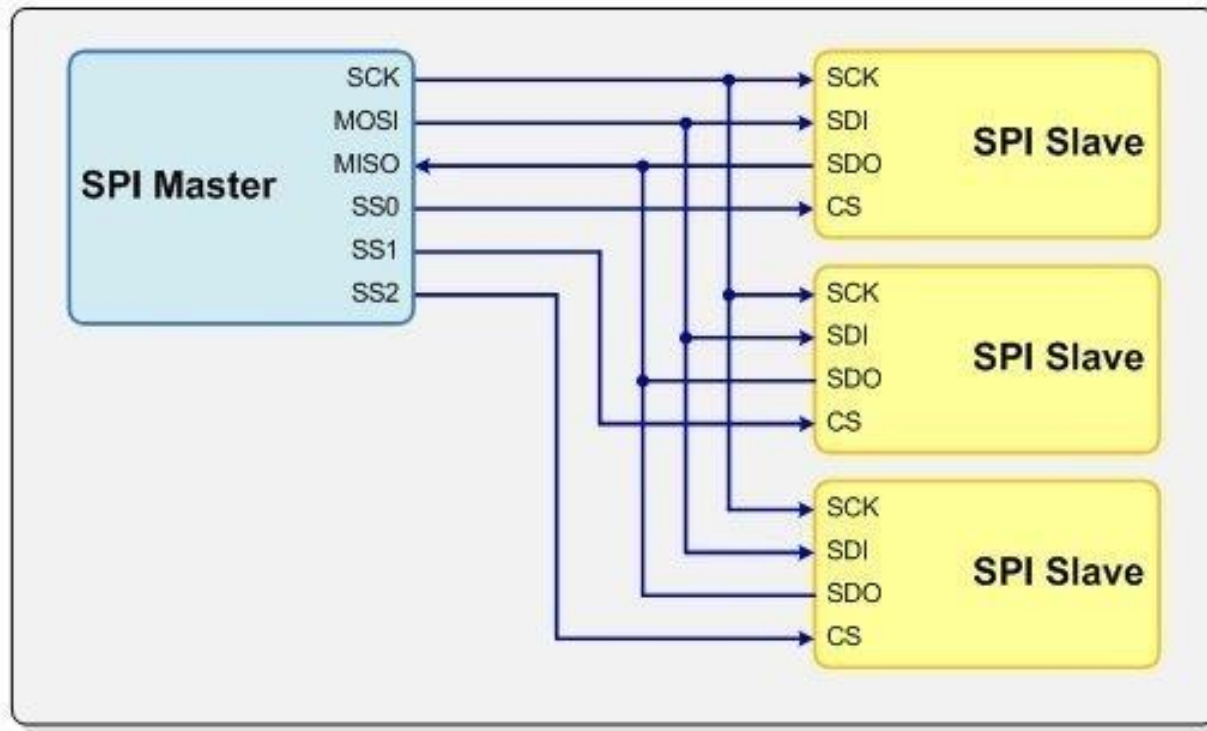
4.1 SPI

- ❑ Serial Peripherals Interface, very popular protocol to connect highspeed peripherals/devices on the same PCB.
 - | Thẻ nhớ, chip nhớ flash memory, MEM sensors, displays,...
- ❑ Synchronous, full duplex, serial data communication.
- ❑ Two types of devices: Master and Slave.
- ❑ Four signals:
 - | MOSI (Master Out Slave In).
 - | MISO (Master In Slave Out).
 - | SCLK (Serial Clock): generated by Master.
 - | CS/SS (Chip Select, Slave Select).
- ❑ Some slaves also use signal names SDI, SDO, SCK.



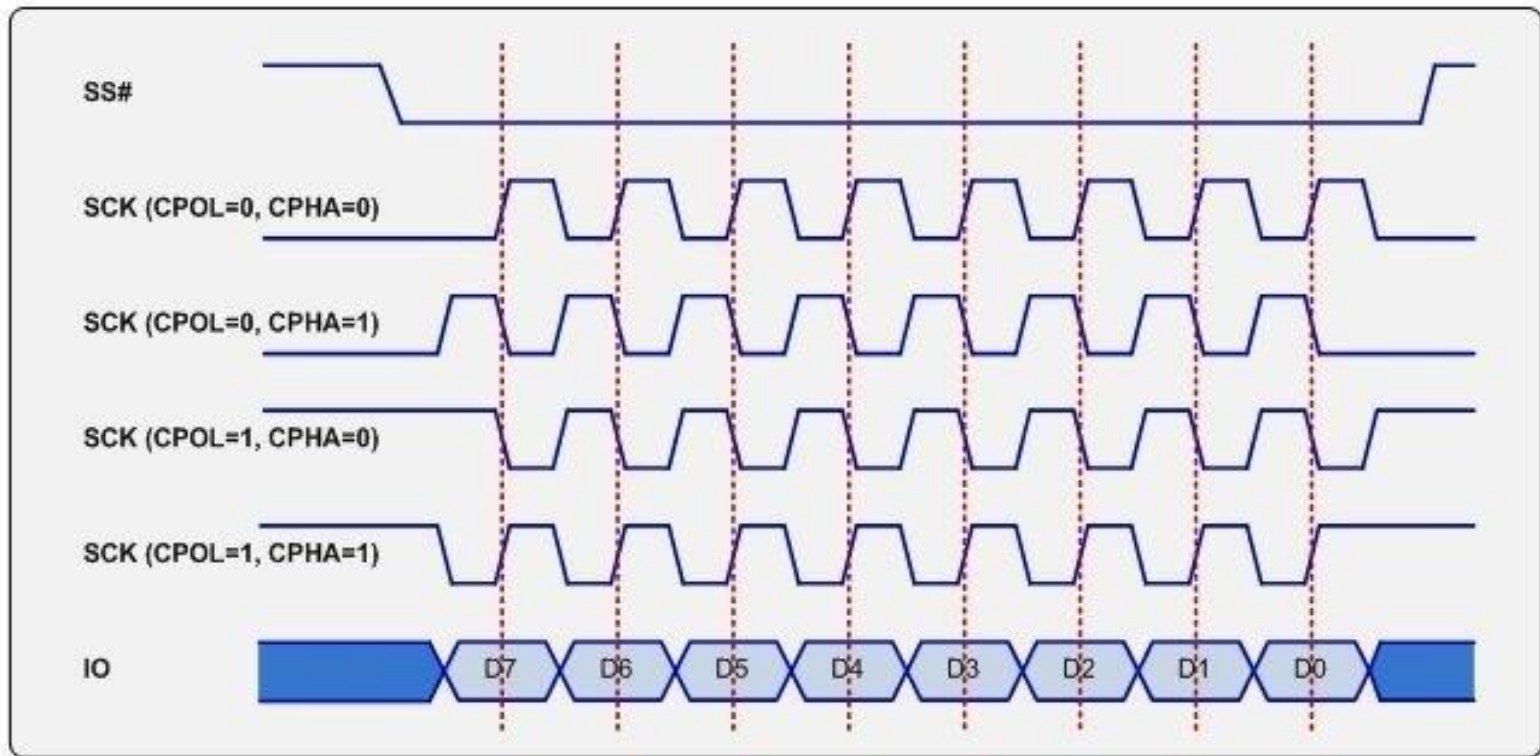
SPI config

❑ One Master with multiple Slaves



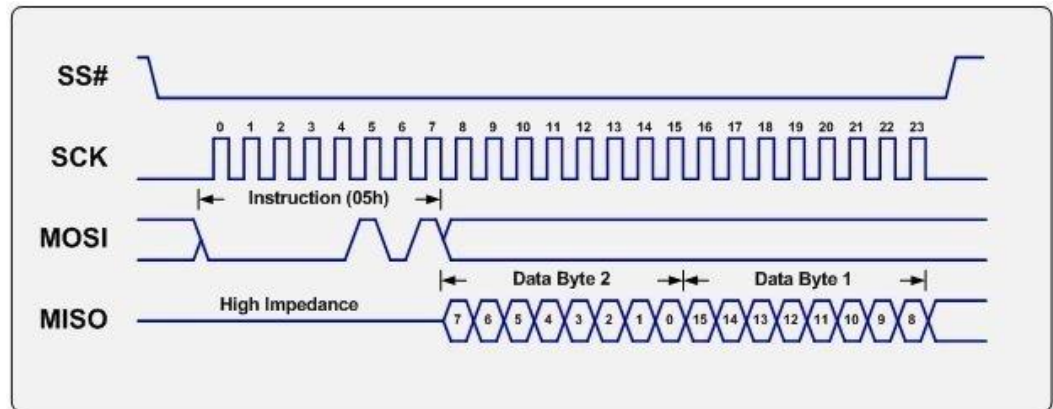
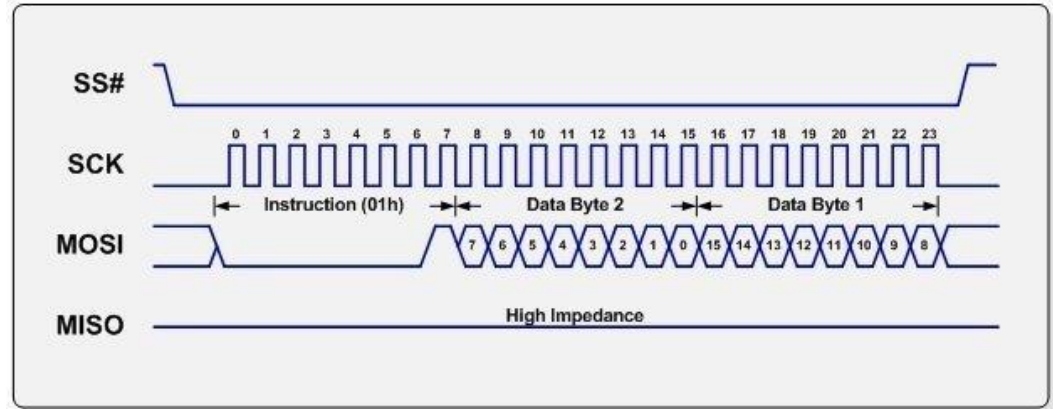
SPI mode

- ❑ 4 modes: Mode 0 – Mode 3
- ❑ Data are sent/received at edges of Serial Clock (SCK)



SPI transaction

- ❑ Master sends command
- ❑ Master sends data (write transaction) or
- ❑ Master reads data (read transaction)



Programming with SPI on STM32F4

❑ Exercise: Check out the following functions from UM1725

HAL_SPI_Init()

HAL_SPI_DeInit()

HAL_SPI_MspltInit()

HAL_SPI_MspDeInit()

HAL_SPI_RegisterCallback()

HAL_SPI_UnRegisterCallback()

HAL_SPI_Transmit()

HAL_SPI_Receive()

HAL_SPI_TransmitReceive()

HAL_SPI_Transmit_IT()

HAL_SPI_Receive_IT()

HAL_SPI_TransmitReceive_IT()

HAL_SPI_Transmit_DMA()

HAL_SPI_Receive_DMA()

HAL_SPI_TransmitReceive_DMA()

HAL_SPI_Abort()

HAL_SPI_Abort_IT()

HAL_SPI_DMABase()

HAL_SPI_DMAResume()

HAL_SPI_DMAStop()

HAL_SPI_IRQHandler()

HAL_SPI_TxCpltCallback()

HAL_SPI_RxCpltCallback()

HAL_SPI_TxRxCpltCallback()

HAL_SPI_TxHalfCpltCallback()

HAL_SPI_RxHalfCpltCallback()

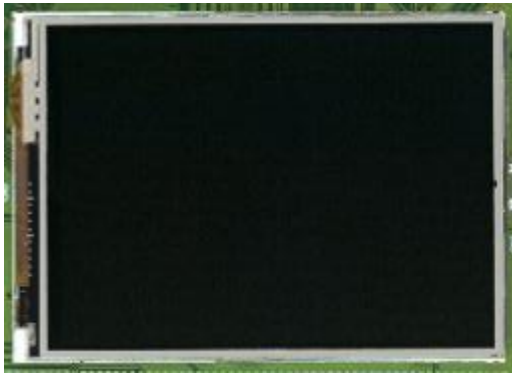
HAL_SPI_TxRxHalfCpltCallback()

HAL_SPI_ErrorCallback()

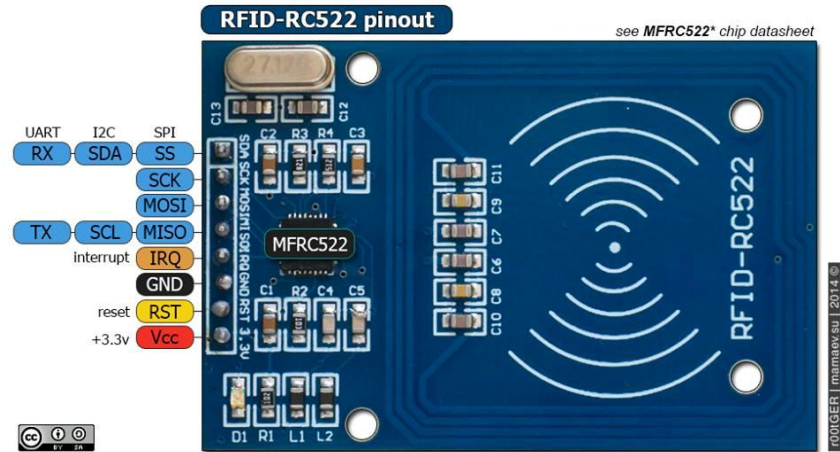
HAL_SPI_AbortCpltCallback()

Example

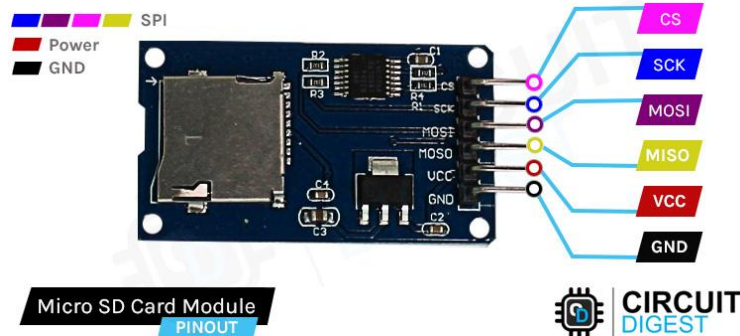
❑ Some peripherals with SPI interface



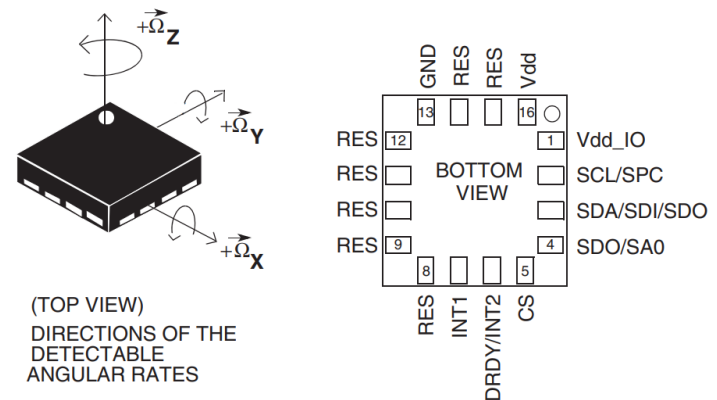
ILI9431 Graphics LCD Controller



RC522 RFID controller



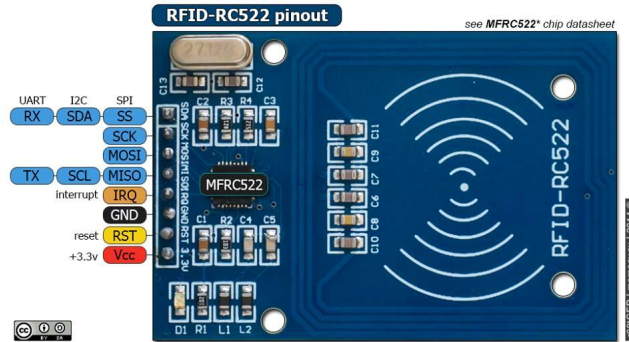
Micro SD card



L3GD20 MEMS gyroscope

Ex 1: Interfacing with RC522 RFID reader

- ❑ RC522: RFID 13.56 MHz IC, with SPI/I2C/UART interfaces
→ The given module only works with SPI



RC522 RFID module



RFID 13.56 MHz tags

- ❑ Hardware: connect RC522 with STM32F429's SPI4
 - | **SS** → **PE4** config via GPIO
 - | **SCK** → **PE2 (SPI4_SCK)** config via SPI4 in Connectivity
 - | **MISO** → **PE5 (SPI4_MISO)**
 - | **MOSI** → **PE6 (SPI4_MOSI)**

Ex 1: Interfacing with RC522 RFID reader

- ❑ Create a new project and config the following params
 - | GPIO: PE4 with Output Push-pull
 - | USART1: Asynchronous 115 kbps
 - | SPI4: Full-Duplex Master

The image displays three screenshots from the STM32CubeMX IDE, illustrating the configuration of SPI4 and USART1 for interfacing with an RC522 RFID reader.

Left Screenshot: Pinout & Configuration

- The **Pinout & Configuration** tab is active.
- The **Categories** dropdown is set to **A->Z**.
- The **Connectivity** list on the left shows **SPI4** selected with a green checkmark.
- The **SPI4 Mode and Configuration** panel shows:
 - Mode:** Full-Duplex Master
 - Hardware NSS Signal:** Disable
 - Configuration** section includes buttons for **DMA Settings**, **GPIO Settings**, **User Constants**, and **NVIC Settings**.
 - Parameter Settings** is selected.
 - Basic Parameters:** Frame Format (Motorola), Data Size (8 Bits), First Bit (MSB First).
 - Clock Parameters:** Prescaler (for Bau. 16), Baud Rate (1000.0 KBits/s), Clock Polarity (Low).
- The **Pinout** section on the right shows the pin assignments:
 - SPI4_SCK:** PE2
 - GPIO_Output:** PE4
 - SPI4_MISO:** PE5
 - SPI4_MOSI:** PE6

Middle Screenshot: Categories

- The **Categories** dropdown is set to **A->Z**.
- The list of components shows **SPI4** selected with a green checkmark.
- Other components listed include SPI1, SPI2, SPI3, SPI5, SPI6, UART4, UART5, UART7, UART8, USART1, USART2, USB_OTG_FS, and USB_OTG_HS.

Right Screenshot: USART1 Mode and Configuration

- The **USART1 Mode and Configuration** panel is shown.
- Mode:** Asynchronous
- Hardware Flow Control (RS232):** Disable
- Configuration** section includes buttons for **Reset Configuration**, **NVIC Settings**, **DMA Settings**, **GPIO Settings**, **Parameter Settings**, and **User Constants**.
- Parameter Settings** is selected.
- Basic Parameters:** Baud Rate (115200 Bits/s), Word Length (8 Bits (including Parity)), Parity (None), Stop Bits (1).

Ex 1: Interfacing with RC522 RFID reader

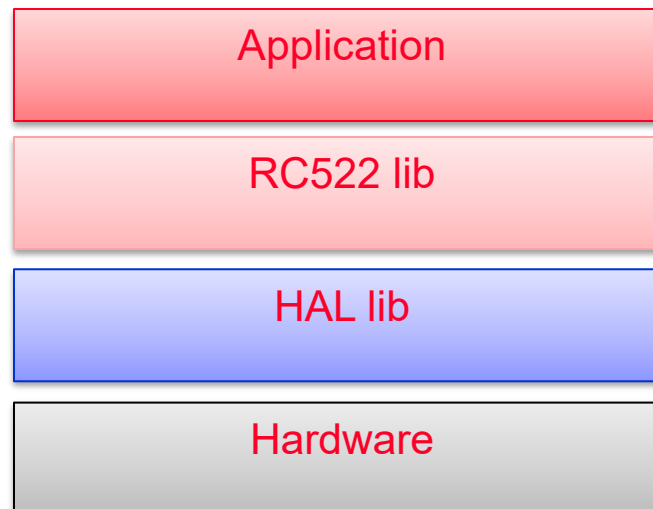
- ❑ Add the files `tm_stm32f4_mfrc522.*` to project.
- ❑ Add source code to `main()`.
- ❑ Run and check output on PC's terminal (Hercules).

```
char buf[100];
TM_MFRC522_Init();
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    uint8_t CardID[5];
    HAL_Delay(100);
    if (TM_MFRC522_Check(CardID) == MI_OK) {
        sprintf (buf, "%s", "RFID Card Found!\r\n");
        HAL_UART_Transmit(&huart1,
                          (const uint8_t*) buf, strlen(buf), 2);
        HAL_Delay(300);
    }
    else{
        sprintf (buf, "%s", "RFID Card Not Found!\r\n");
        HAL_UART_Transmit(&huart1,
                          (const uint8_t*) buf, strlen(buf), 2);
        HAL_Delay(300);
    }
}
```

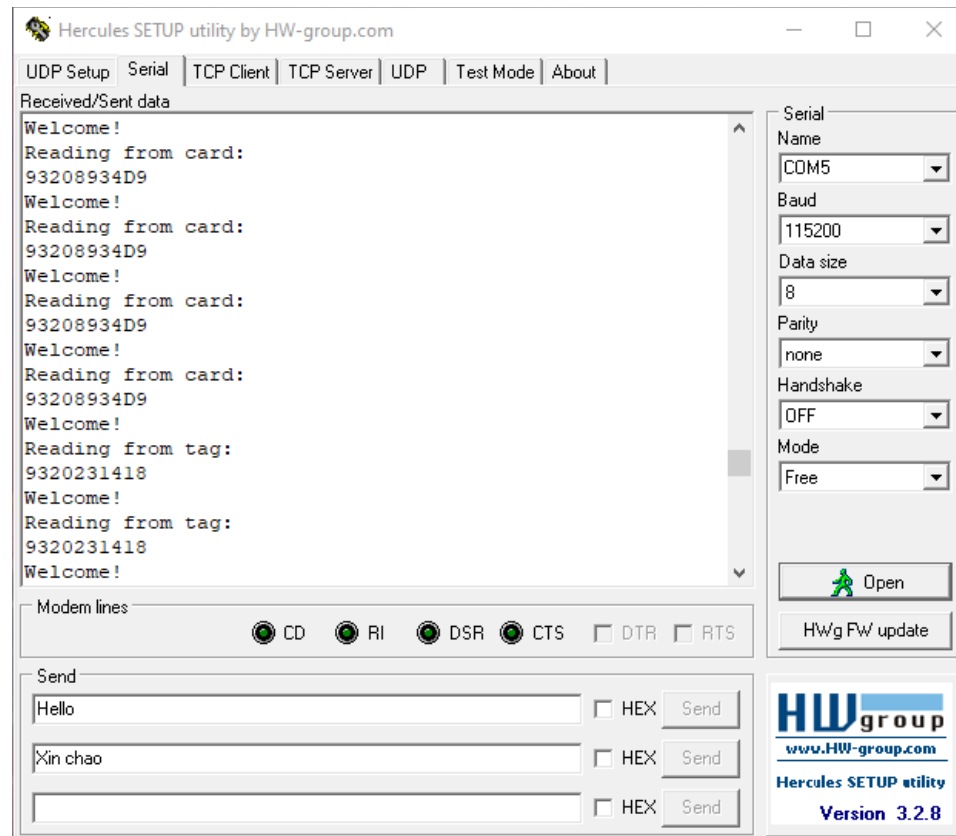
Ex 1: Interfacing with RC522 RFID reader - Explanation

- ❑ Hardware: STM32F4 + RC522
- ❑ HAL lib (SPI driver): low level functions to provide low level data read/write via SPI between MCU and RC522.
- ❑ RC522 lib (RC522 driver): basic function to provide high level interface with RC522 such as tag detection, tag data reading (tag data writing is not supported).
- ❑ Application: uses RC522 lib to build end-users' application.



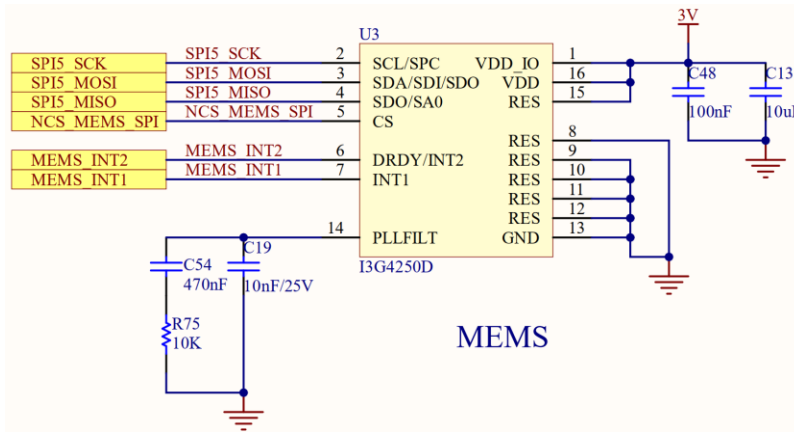
Ex 1: Interfacing with RC522 RFID reader

- ❑ Add your own code to read the RFID tag and sent to PC via USART1.

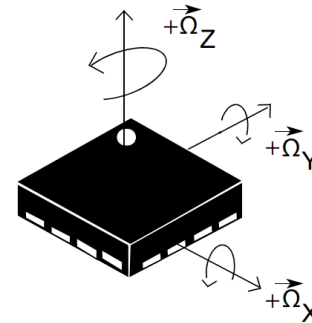


Ex 2: Interfacing with I3G4250D

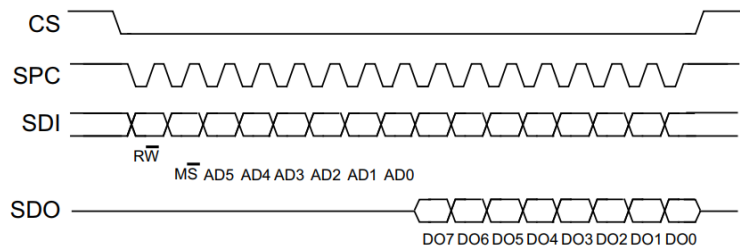
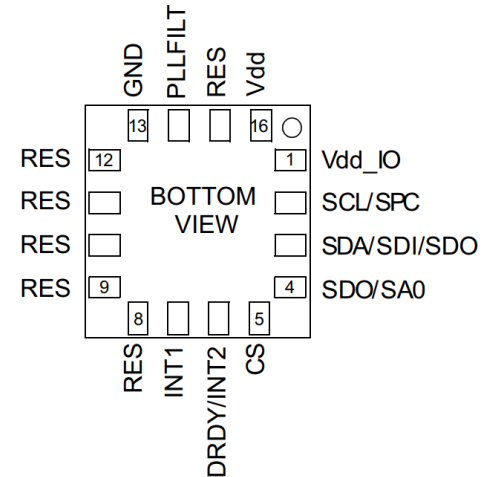
- I3G4250D is the gyroscope sensor already soldered on STM32F429 board, connected to MCU via SPI5.



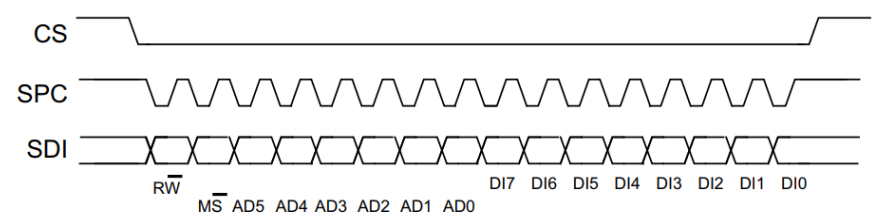
Schematic



(TOP VIEW)
DIRECTIONS OF THE
DETECTABLE
ANGULAR RATES



SPI read



SPI write

Ex 2: Interfacing with I3G4250D

- ❑ Write code on STM32F429 to read angular rate (AngleX, AngleY, AngleZ) then
 - | Send to PC via USART1
 - | LED3 ON if AngleX > 0, OFF otherwise.
 - | LED4 ON if AngleY > 0, OFF otherwise.

Ex 2: Interfacing with I3G4250D

❑ Hardware:

- | Create new project, enable RCC and CPU clock to 180 MHz
- | Configure USART1, PC1, and SPI5

The screenshot displays the STM32CubeMX software interface for configuring the SPI5 peripheral. On the left, the 'Connectivity' tab is active, showing a list of peripherals where SPI5 is selected with a green checkmark. The main panel, titled 'SPI5 Mode and Configuration', is divided into 'Mode' and 'Configuration' sections. In the 'Mode' section, 'Full-Duplex Master' is selected for the Mode and 'Disable' for the Hardware NSS Signal. The 'Configuration' section includes a 'Reset Configuration' button and tabs for 'NVIC Settings', 'DMA Settings', 'GPIO Settings', 'Parameter Settings' (which is currently active), and 'User Constants'. Below these tabs, a search bar is present, followed by expandable sections for 'Basic Parameters', 'Clock Parameters', and 'Advanced Parameters'. The 'Basic Parameters' section shows 'Frame Format' as Motorola, 'Data Size' as 8 Bits, and 'First Bit' as MSB First. The 'Clock Parameters' section shows 'Prescaler (for Baud Rate)' as 16, 'Baud Rate' as 5.625 MBits/s, 'Clock Polarity (CPOL)' as Low, and 'Clock Phase (CPHA)' as 1 Edge. The 'Advanced Parameters' section shows 'CRC Calculation' as Disabled and 'NSS Signal Type' as Software. To the right of the configuration panel, a pinout diagram shows the physical pins of the microcontroller. The pins are color-coded: green for digital I/O, yellow for power, and grey for other functions. A black arrow points to pin PC1, which is labeled 'GPIO_Output'.

Peripheral	Pin
SPI5_SCK	PF7
SPI5_MISO	PF8
SPI5_MOSI	PF9
RCC_OSC_IN	PH0/..
RCC_OSC_OUT	PH1/..
GPIO_Output	PC1

Ex 2: Interfacing with L3G4250D

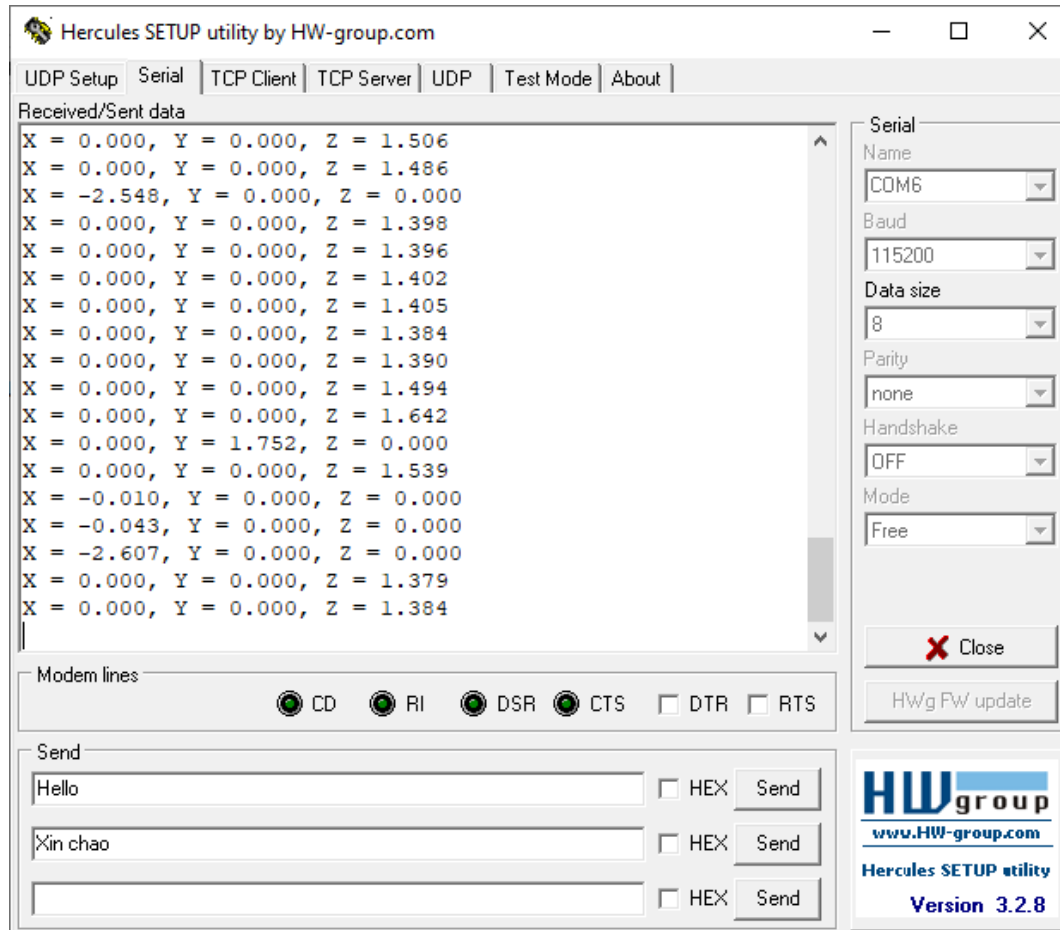
❑ Software:

- | Add gyroscope lib (**L3GD20**) to project.
- | Add code to main().
- | Run and check output.

```
95  L3GD20_Init();
96  /* USER CODE END 2 */
97
98  /* Infinite loop */
99  /* USER CODE BEGIN WHILE */
100 while (1)
101 {
102     /* USER CODE END WHILE */
103     /* USER CODE BEGIN 3 */
104     L3GD20_loop();
105     HAL_Delay(1);
106     float X = get_Angle_X(); float Y = get_Angle_Y(); float Z = get_Angle_Z();
107
108     if (X > 0 && Y > 0 && Z > 0) {
109         for (int i = 0; i < 5; i++) {
110             HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
111             HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
112             HAL_Delay(1000);
113         }
114         X = 0; Y = 0; Z = 0;
115     }
116     else {
117         HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
118         HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
119         X = 0; Y = 0; Z = 0;
120     }
121 }
```

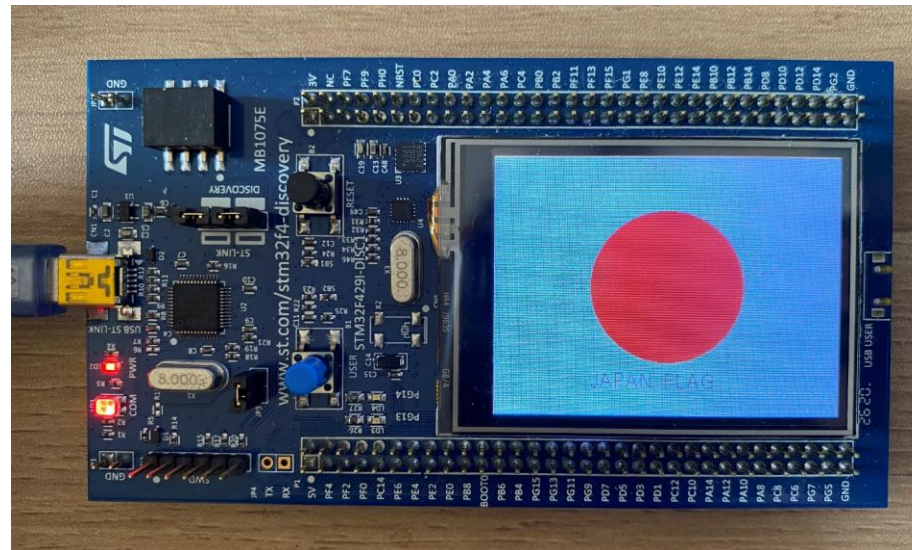
Ex 2: Interfacing with I3G4250D

- ❑ Add code to main to continuously send angular rate readings to PC



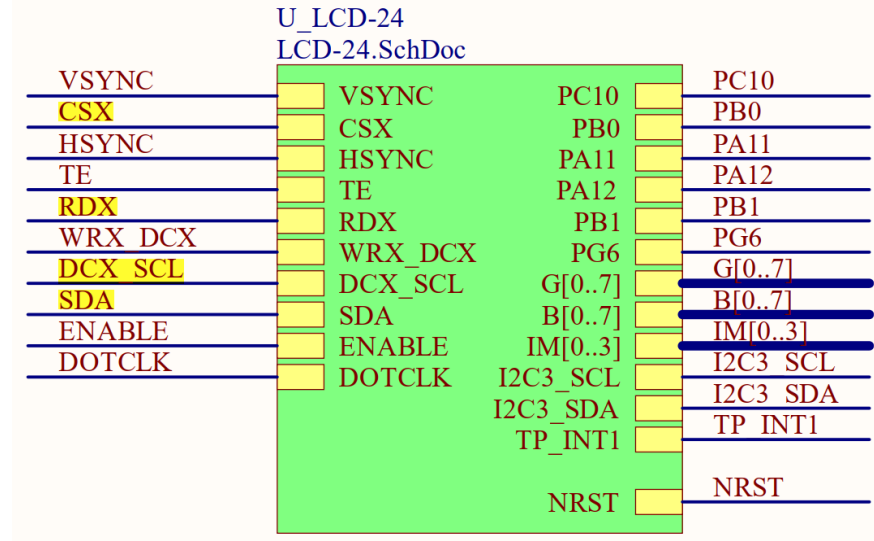
Ex 3: Interfacing with built-in touch LCD

- ❑ STM32F429I-DISC board has a built-in touch LCD.
 - | LCD: 240x320 graphics driven by the ILI9341 LCD controller (SPI interface).
 - | Touch: driven by the STMPE811 resistive touch controller (I2C) interface.
- ❑ Sample code: interface with the ILI9341 controller via SPI and draw the Japanese flag.



Ex 3: Interfacing with built-in touch LCD

- ❑ Create new project and configure HCLK to 180 MHz
- ❑ Enable the SPI5 on STM32F429
- ❑ ILI9341 connection
 - | SS → PC2
 - | DC → PD13
 - | RST → PD12
 - | SCK → PF7
 - | MISO → PF8 (off)
 - | MOSI → PF9



PA4	VSYNC	VSYNC
PC2	CSX	CSX
PC6	HSYNC	HSYNC
PD11	TE	TE
PD12	RDX	RDX
PD13	WRX_DCX	WRX_DCX
PF7	DCX_SCL	DCX_SCL
PF9	SDA	SDA
PF10	ENABLE	ENABLE
PG7	DOTCLK	DOTCLK
PA8	I2C3_SCL	I2C3_SCL
PC9	I2C3_SDA	I2C3_SDA
PA15	TP_INT1	TP_INT1
PA7	ACP_RST	ACP_RST

Ex 3: Interfacing with built-in touch LCD

The image shows the STM32CubeMX configuration interface for a microcontroller. The left sidebar lists various peripherals, with SPI5 selected. The main configuration area shows the SPI5 configuration, with the Mode set to 'Transmit Only Master' and Hardware NSS Signal set to 'Disable'. The Configuration tab is active, showing the Parameter Settings sub-tab. The Basic Parameters section shows the Frame Format as Motorola, Data Size as 8 Bits, and First Bit as MSB First. The Clock Parameters section shows the Prescaler (for Baud Rate) as 4, Baud Rate as 22.5 Mbits/s, Clock Polarity (CPOL) as Low, and Clock Phase (CPHA) as 1 Edge. The Advanced Parameters section shows CRC Calculation as Disabled and NSS Signal Type as Software. To the right of the configuration area, a pin list shows the connections for SPI5_SCK (PF7), SPI5_MOSI (PF9), RCC_OSC_IN (PH0), and RCC_OSC_OUT (PH1). The pin list also shows the connections for GPIO_Output (PD13, PD12, and PC2). A detailed view of the DMA configuration is shown at the bottom, with the DMA Request set to SPI5_TX, Stream set to DMA2 Stream 4, Direction set to Memory To Perip..., and Priority set to Low.

Mode: Transmit Only Master
Hardware NSS Signal: Disable

Configuration

Reset Configuration

✓ NVIC Settings ✓ DMA Settings ✓ GPIO Settings
✓ Parameter Settings ✓ User Constants

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Frame Format: Motorola
Data Size: 8 Bits
First Bit: MSB First

Clock Parameters

Prescaler (for Baud Rate): 4
Baud Rate: 22.5 Mbits/s
Clock Polarity (CPOL): Low
Clock Phase (CPHA): 1 Edge

Advanced Parameters

CRC Calculation: Disabled
NSS Signal Type: Software

SPI5_SCK: PF7
SPI5_MOSI: PF9
RCC_OSC_IN: PH0/..
RCC_OSC_OUT: PH1/..
NRST
PC0
PC1
GPIO_Output: PC2
PC3

VSS
VDD
VSS
PD13: GPIO_Output
PD12: GPIO_Output
PD11

Reset Configuration

✓ NVIC Settings ✓ DMA Settings ✓ GPIO Settings
✓ Parameter Settings ✓ User Constants

DMA Request	Stream	Direction	Priority
SPI5_TX	DMA2 Stream 4	Memory To Perip...	Low

Ex 3: Interfacing with built-in touch LCD

- ❑ Add the ILI9341 driver.
- ❑ Add some below code to main().
- ❑ Run and see the flag of Japan.
 - | To-do: draw the flag of Vietnam/France.

```
93  /* Initialize all configured peripherals */
94  MX_GPIO_Init();
95  MX_DMA_Init();
96  MX_SPI5_Init();
97  /* USER CODE BEGIN 2 */
98  ILI9341_Init();
99  ILI9341_FillScreen(WHITE);
100 ILI9341_SetRotation(SCREEN_HORIZONTAL_2);
101 ILI9341_DrawText("JAPAN FLAG", FONT4, 90, 200, BLACK, WHITE);
102 ILI9341_DrawFilledCircle(160, 120, 50, RED);
103 HAL_Delay(1000);
104 /* USER CODE END 2 */
```

4.2 I²C

❑ Disadvantages of SPI:

- | Require many GPIO pins.
- | Difficult to interface multiple peripherals at the same time.

➔ Inter-Integrated Circuit (I2C): designed by Philips Semiconductor for interfacing multiple ICs using the same bus.

- | Less GPIO pins.
- | Flexible and high speed.

❑ Examples: display, sensor,...

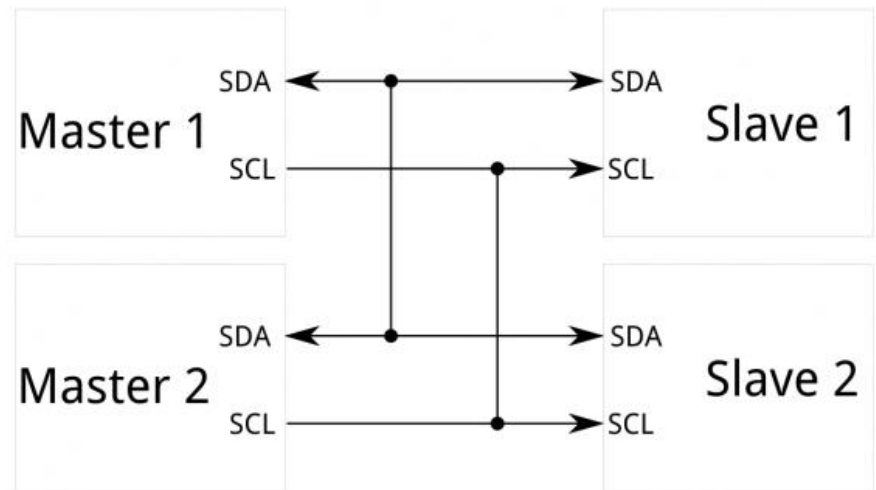
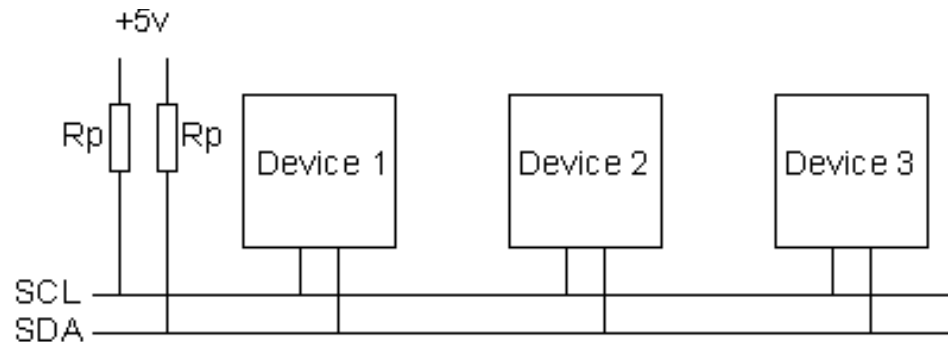
I²C bus

❑ Bus topology

- | SCL: serial clock
- | SDA: serial data

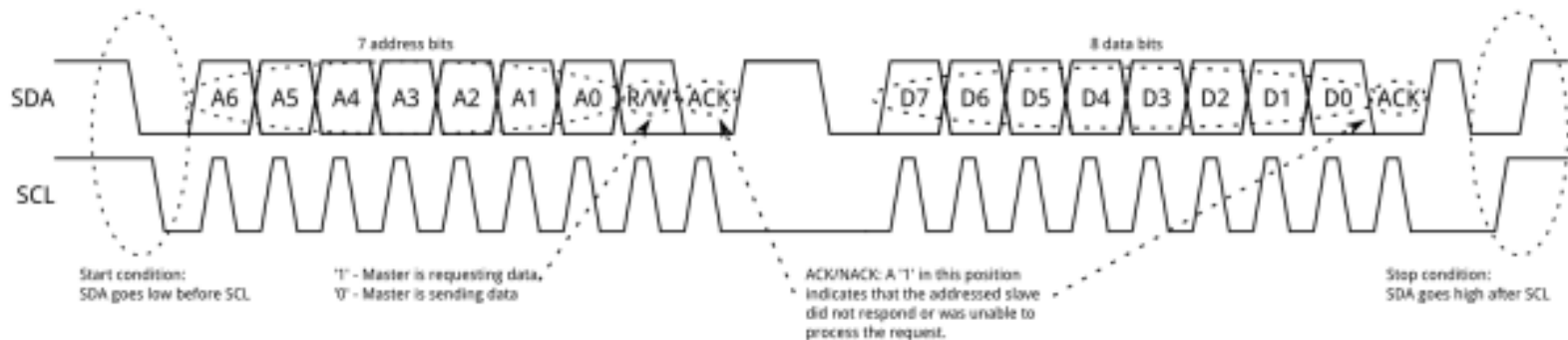
❑ Multiple slaves

❑ Even multiple masters



I2C communication protocol

- ❑ Master initialize the transaction
 - | Start bit
 - | Address frame: 7 or 10 bit địa chỉ + R/W bit
 - | Data frame: 8 bit (multiple bytes can be sent/received continuously)
 - | Stop bit
- ❑ The slave with matching address will do the required operation (read/write) then send response.



Programming with SPI on STM32F4

❑ Check out the following functions from UM1725

<i>HAL_I2C_Init()</i>	<i>HAL_I2C_Master_Transmit()</i>	<i>HAL_I2C_IsDeviceReady()</i>
<i>HAL_I2C_DeInit()</i>	<i>HAL_I2C_Master_Receive()</i>	<i>HAL_I2C_Master_Seq_Transmit_IT()</i>
<i>HAL_I2C_MspInit()</i>	<i>HAL_I2C_Slave_Transmit()</i>	<i>HAL_I2C_Master_Seq_Transmit_DMA()</i>
<i>HAL_I2C_MspDeInit()</i>	<i>HAL_I2C_Slave_Receive()</i>	<i>HAL_I2C_Master_Seq_Receive_IT()</i>
	<i>HAL_I2C_Master_Transmit_IT()</i>	<i>HAL_I2C_Master_Seq_Receive_DMA()</i>
<i>HAL_I2C_RegisterCallback()</i>	<i>HAL_I2C_Master_Receive_IT()</i>	<i>HAL_I2C_Slave_Seq_Transmit_IT()</i>
<i>HAL_I2C_UnRegisterCallback()</i>	<i>HAL_I2C_Slave_Transmit_IT()</i>	<i>HAL_I2C_Slave_Seq_Transmit_DMA()</i>
<i>HAL_I2C_RegisterAddrCallback()</i>	<i>HAL_I2C_Slave_Receive_IT()</i>	<i>HAL_I2C_Slave_Seq_Receive_IT()</i>
<i>HAL_I2C_UnRegisterAddrCallback()</i>	<i>HAL_I2C_Master_Transmit_DMA()</i>	<i>HAL_I2C_Slave_Seq_Receive_DMA()</i>
	<i>HAL_I2C_Master_Receive_DMA()</i>	<i>HAL_I2C_EnableListen_IT()</i>
	<i>HAL_I2C_Slave_Transmit_DMA()</i>	<i>HAL_I2C_DisableListen_IT()</i>
	<i>HAL_I2C_Slave_Receive_DMA()</i>	<i>HAL_I2C_Master_Abort_IT()</i>
	<i>HAL_I2C_Mem_Write()</i>	
	<i>HAL_I2C_Mem_Read()</i>	
	<i>HAL_I2C_Mem_Write_IT()</i>	
	<i>HAL_I2C_Mem_Read_IT()</i>	
	<i>HAL_I2C_Mem_Write_DMA()</i>	
	<i>HAL_I2C_Mem_Read_DMA()</i>	

Programming with SPI on STM32F4

❑ Example: data transmit (locking mode)

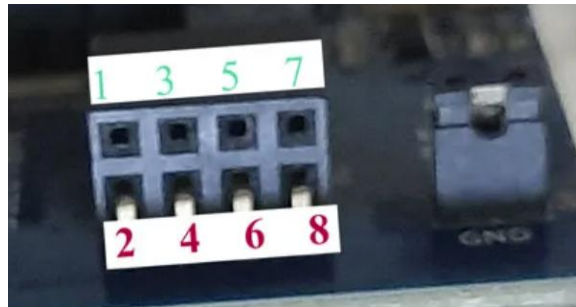
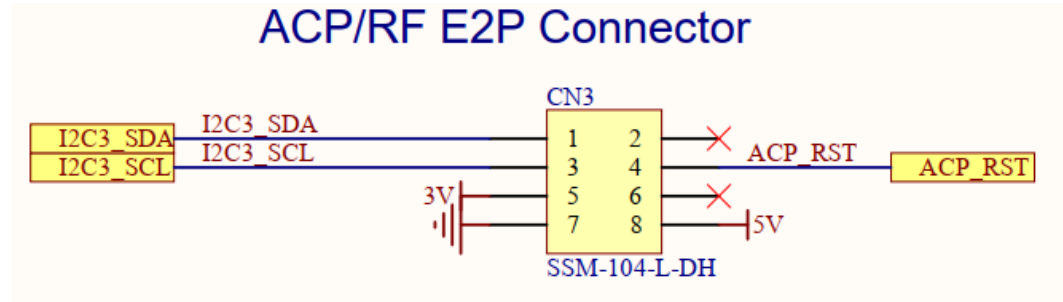
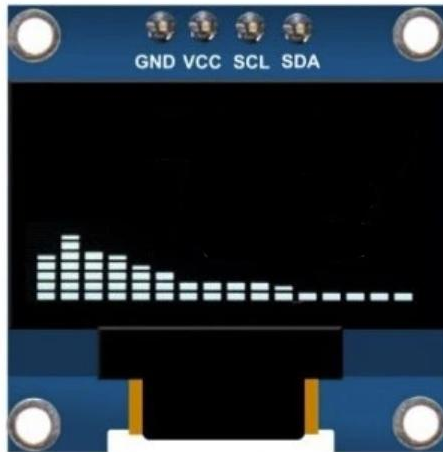
```
HAL_StatusTypeDef HAL_I2C_Master_Transmit (I2C_HandleTypeDef * hi2c,  
                                             uint16_t DevAddress,  
                                             uint8_t * pData,  
                                             uint16_t Size,  
                                             uint32_t Timeout)
```

❑ Note:

- | DevAddress: 8 bit integer with **7 msb as device address**, and the lsb as the R/W bit.
 - R/W = 0: write transaction (master writes data to slave)
 - R/W = 1: read transaction (master reads data from slave)

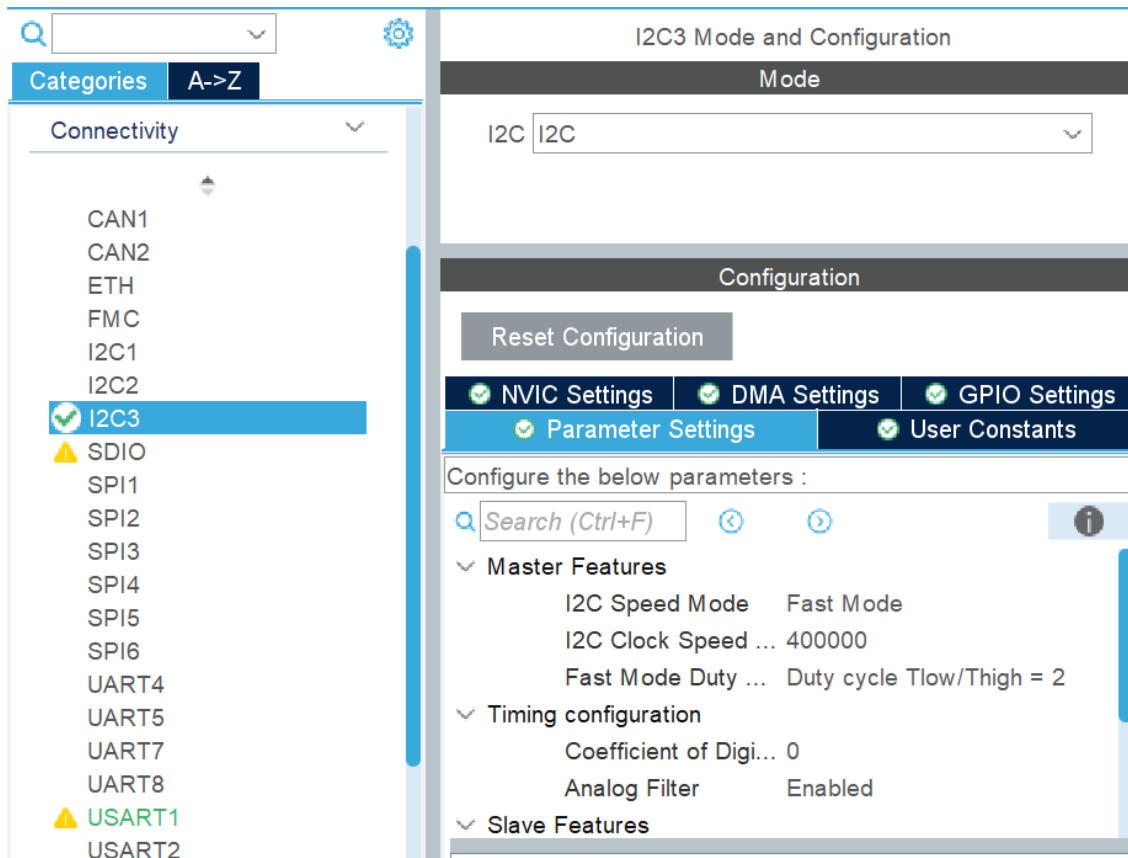
Ex: interfacing with OLED SH1106

- ❑ SH1106: OLED monochrome graphics display module with 128x64 resolution.
- ❑ Supports I2C at address 0x3C.
- ❑ Hardware: plug the SH1106 module to CN2



Ex: interfacing with OLED SH1106

- ❑ Create new project, configure RCC and HCLK 180 MHz
- ❑ Configure the I2C3 (which is connected to CN2)



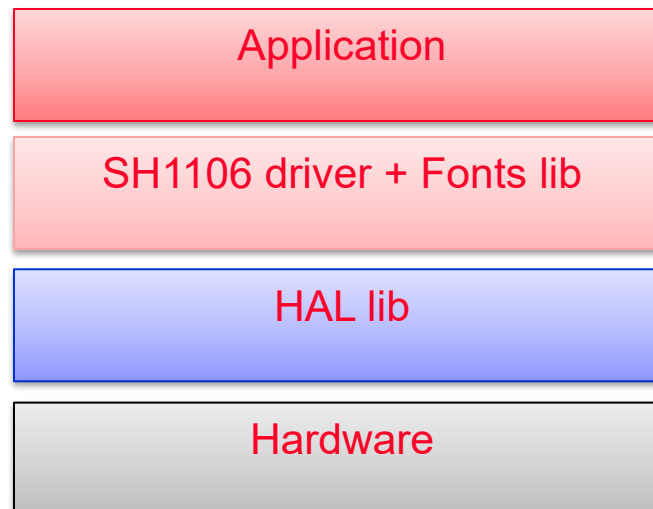
Ex: interfacing with OLED SH1106

- ❑ Add the files sh1106.* and fonts.* to project.
- ❑ Add code to main().
- ❑ Run and check display content on SH1106.

```
94  /* Initialize all configured peripherals */
95  MX_GPIO_Init();
96  MX_I2C3_Init();
97  MX_USART1_UART_Init();
98  /* USER CODE BEGIN 2 */
99  char buf[100];
100 int X = 0, Y = 0;
101 SH1106_Init ();
102 sprintf (buf, "%s", "Hello TN66");
103 SH1106_GotoXY (12,10); // goto 10, 10
104 SH1106_Puts(buf, &Font_11x18, 1);
105 SH1106_UpdateScreen(); // update screen
106 HAL_Delay(1000);
107 /* USER CODE END 2 */
```

Ex: Interfacing with RC522 RFID reader - Explanation

- ❑ Hardware: STM32F4 + SH1106
- ❑ HAL lib (I2C driver): low level functions to provide low level data read/write via I2C between STM32F4 and SH1106.
- ❑ SH1106 driver: basic function to provide both low-level functions (turn on/off one pixel) and high level graphics functions.
- ❑ Application: uses SH1106 driver to build end-users' application.

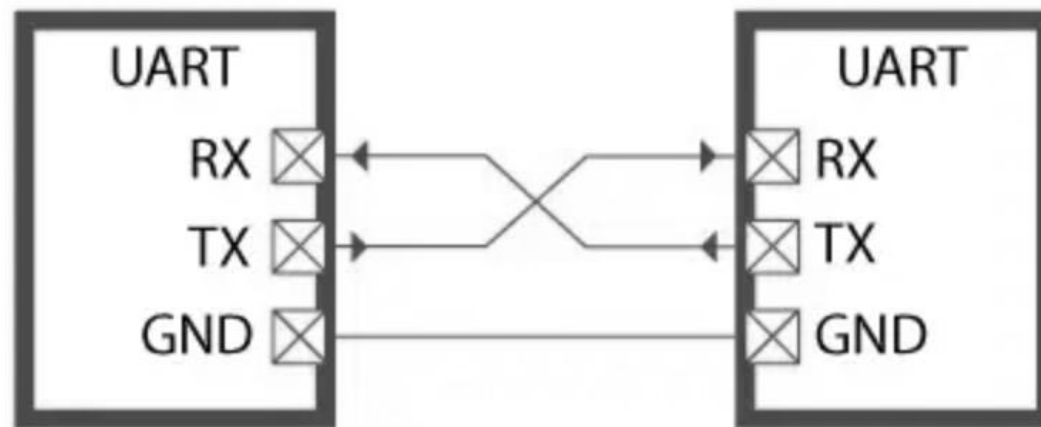


Ex: interfacing with OLED SH1106

- ❑ Use the timer TIM6 to count elapsed time from system start up, then display time on SH1106.
 - | Notes:
 - Do not do anything slow inside any interrupt to avoid system hang because of stack overflow.
 - Separate “back-end” and “front-end”: counting system time on timer 6 interrupt handler, and updating SH1106 on another thread (can be the main loop or another timer interrupt)
- ❑ Scan and display the ID of RFID tag on SH1106.

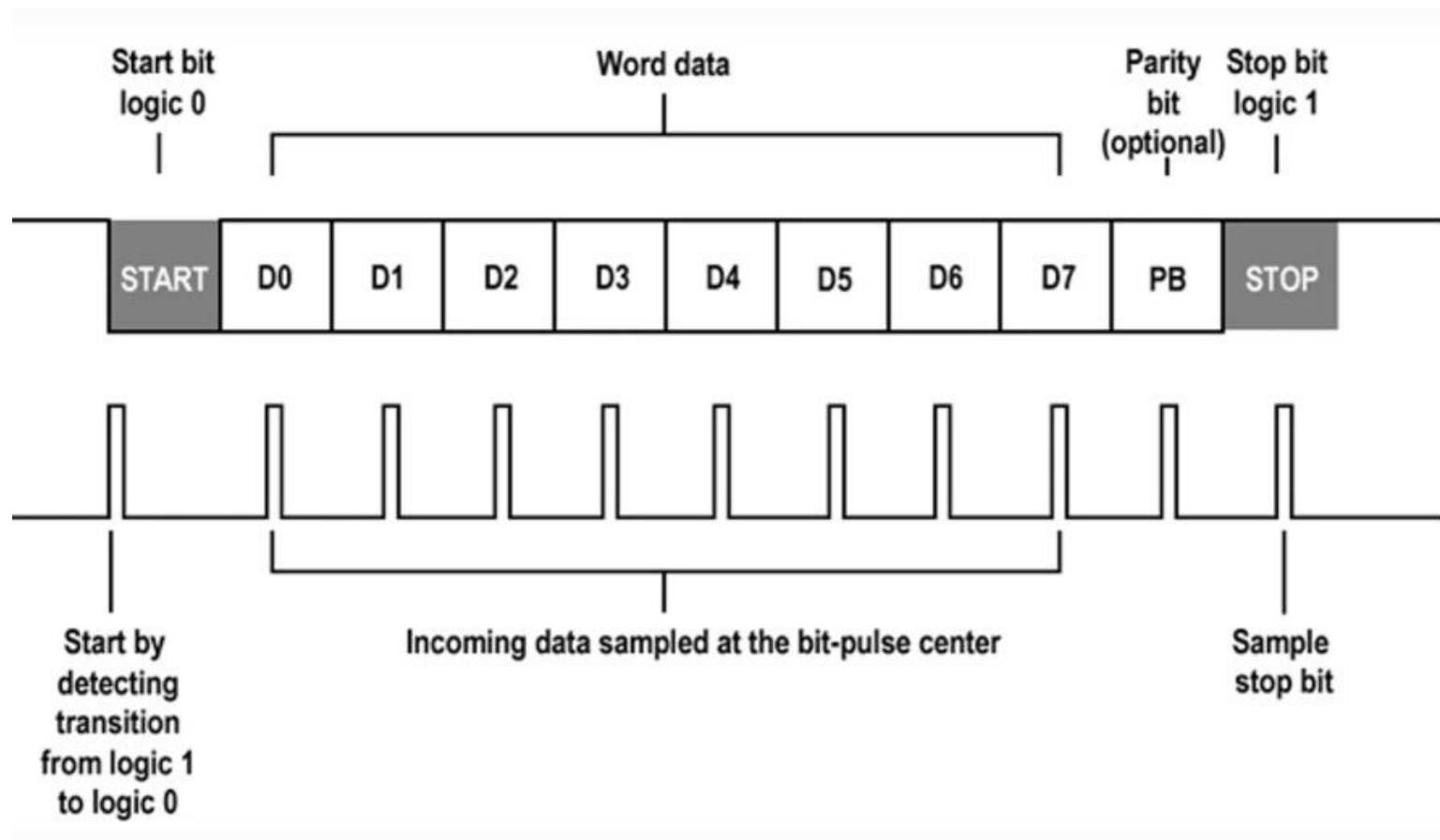
4.3 UART

- ❑ Universal Asynchronous Receiver and Transmitter.
- ❑ Also known as the “serial port” on PC/embedded systems.
- ❑ Two data wires for full duplex transmission.
- ❑ No serial clock:
 - | Transmitter and receiver must work at predefined standard clock rates (baud rate).
 - | Data must be encoded in serial data frame.



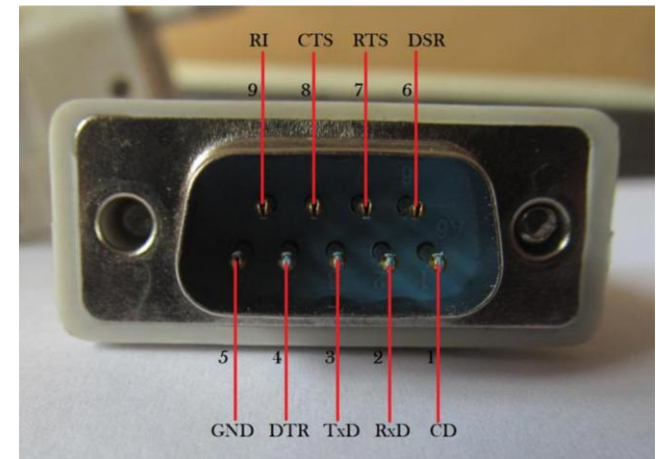
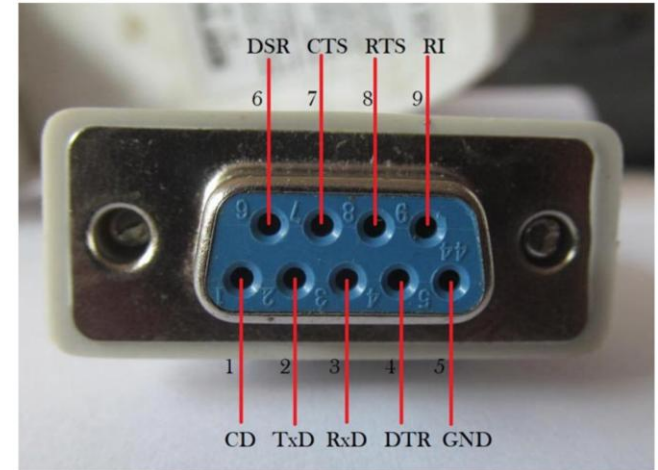
UART data frame

- ❑ 1 start bit, 5-8 data bits, [parity], 1-2 stop bits



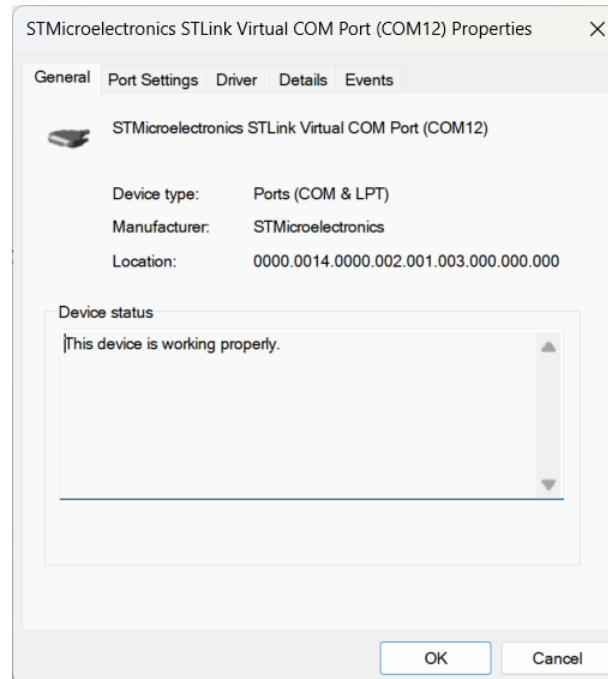
RS232C

- ❑ Electrical standard to for asynchronous serial communication using UART
- ❑ Voltage level:
 - | 0 logic: +3 to +15V
 - | 1 logic : -3 to -15V
- ❑ Baudrate: 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200
- ❑ Besides Rx, Tx, there are 6 control signals.



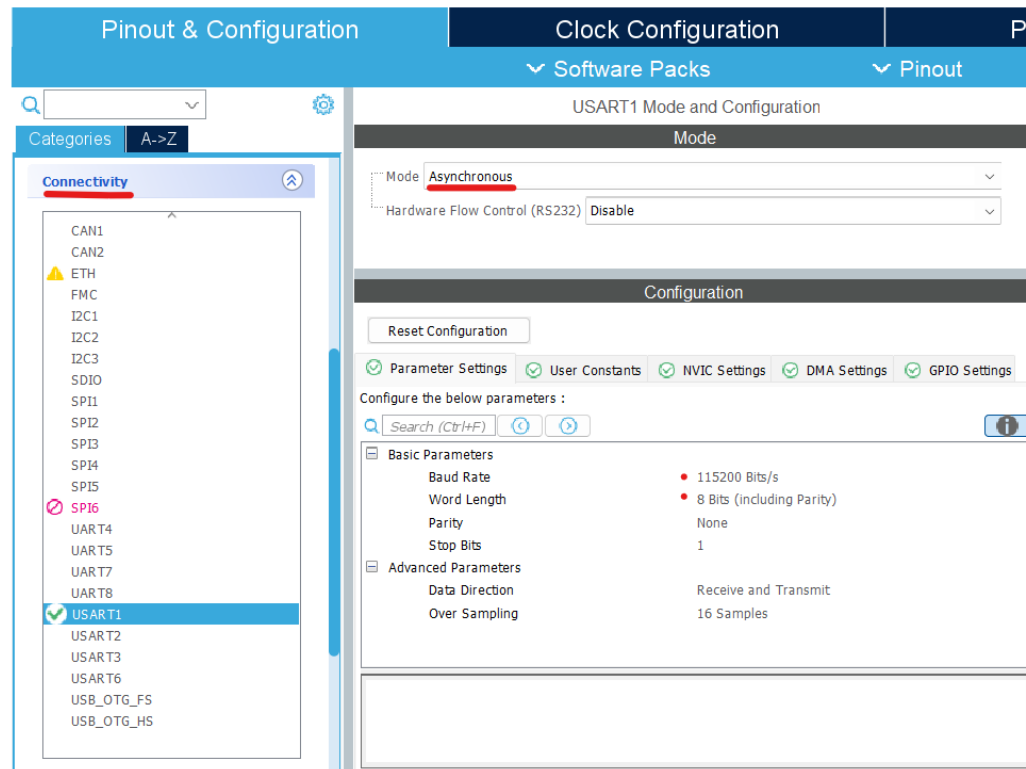
UART on STM32F429

- ❑ STM32F429 supports up to 8x UART/USART ports.
- ❑ UART voltage: 3.3V
- ❑ Note: on the STM32F429-DISC board, the USART1 is connected to the virtual COM on PC via the debugger.



Ex: interfacing STM32F429I with PC via USART1

- ❑ Send the text “Hello” from STM32F429 to PC
 - | Config USART1 in Connectivity
 - | Mode: Asynchronous, Baud rate 115200 bps, Word length 8 bits, Parity none.



Ex: interfacing STM32F429I with PC via USART1

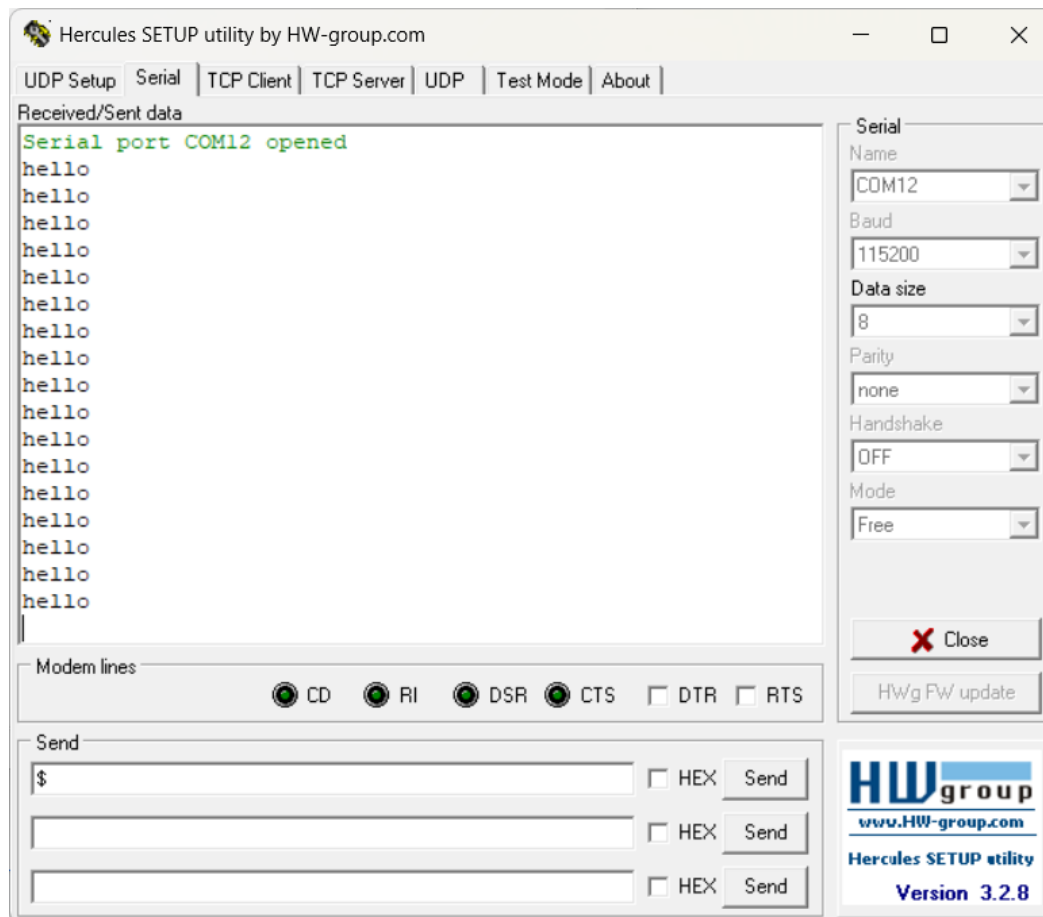
- ❑ Send the text “Hello” from STM32F429 to PC
 - | Config USART1 in Connectivity
 - | Mode: Asynchronous, Baud rate 115200 bps, Word length 8 bits, Parity none.
- ❑ Call HAL_UART_Transmit() to transmit data

```
main.c x  stm32f4xx_it.c  startup_stm32f429zitx.s  InterruptEx.ioc  system_stm32f4xx.c
100      /* Infinite loop */
101      /* USER CODE BEGIN WHILE */
102      while (1)
103      {
104          /* USER CODE END WHILE */
105          char buf[10] = "hello\r\n";
106          HAL_UART_Transmit(&huart1,
107                          (const uint8_t*)buf,
108                          strlen(buf) ,
109                          2);
110          HAL_Delay(500);
111          /* USER CODE BEGIN 3 */
112      }
113      /* USER CODE END 3 */
```

Ex: interfacing STM32F429I with PC via USART1

❑ Run and observe data in Hercules

- | Note: configure Hercules Serial port according to the virtual COM port of the STM32F429I-DISC



Ex: connecting two boards via USART

- ❑ Connect 2 STM32F429I boards by the USART1 on each board
 - | Board 1 – TX (PA9) $\longleftrightarrow$ Board 2 – RX (PA10)
 - | Board 1 – RX (PA10) $\longleftrightarrow$ Board 2 – TX (PA9)
 - | Board 1 – GND $\longleftrightarrow$ Board 2 – GND
- ❑ Write a program on each board (transmitter & receiver)
 - | Send an array of 100 integers from Board 1 to Board 2.
 - | After Board 1 finished sending 100 integers, it turns on LED3.
 - | After Board 2 finished receiving 100 integers, it turns on LED3.

PA9	I/O	FT	-	TIM1_CH2, I2C3_SMBA, USART1_TX, DCMI_D0, EVENTOUT
PA10	I/O	FT	-	TIM1_CH3, USART1_RX, OTG_FS_ID, DCMI_D1, EVENTOUT

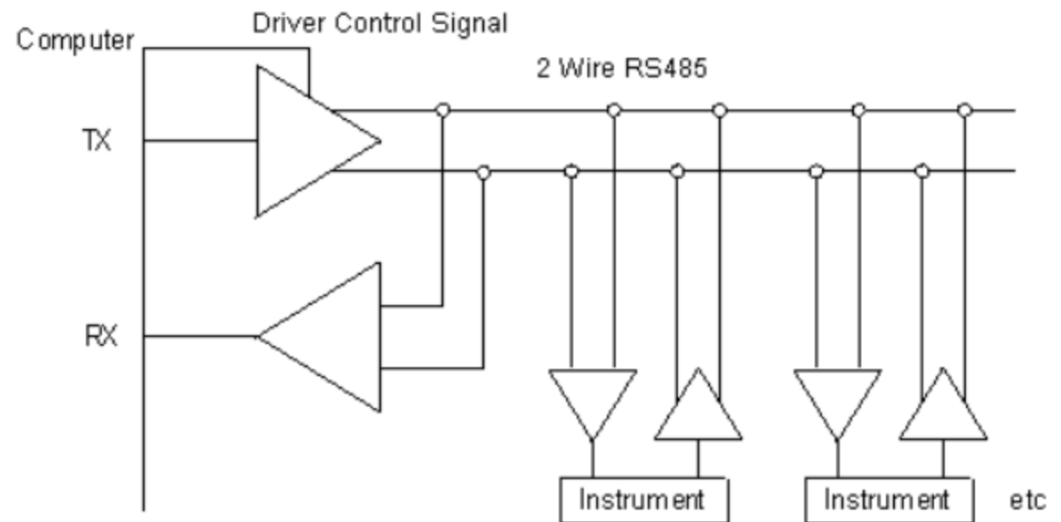
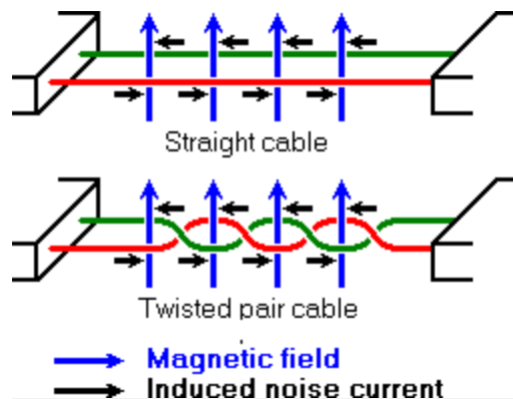
Ex: connecting two boards via USART

The screenshot displays the STM32CubeMX Pinout & Configuration window. The interface is divided into several sections:

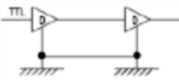
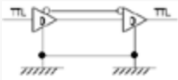
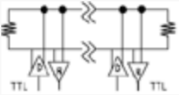
- Pinout & Configuration** (Left Panel):
 - Search bar with a dropdown arrow.
 - Categories: A->Z
 - List of peripherals: SDIO, SPI1, SPI2, SPI3, SPI4, SPI5, SPI6, UART4, UART5, UART7, UART8, **USART1** (selected), USART2, USART3, USART6, USB_OTG_FS, USB_OTG_HS.
 - Multimedia and Security sections with expandable arrows.
- USART1 Mode and Configuration** (Middle Panel):
 - Mode**: Mode set to Asynchronous, Hardware Flow Control (RS232) set to Disable.
 - Configuration**: Reset Configuration button.
 - GPIO Settings**: Enabled (checked).
 - NVIC Settings**: Enabled (checked).
 - DMA Settings**: Enabled (checked).
 - Parameter Settings**: Enabled (checked).
 - User Constants**: Enabled (checked).
 - Configure the below parameters:
 - Search (Ctrl+F) bar.
 - Basic Parameters**: Baud Rate (115200 Bits/s), Word Length (8 Bits (including Parity)), Parity (None).
- Pinout** (Right Panel):
 - Pinout view selected (System view also available).
 - Pinout diagram showing various pins: VDD, VSS, VCAP.., PA13, PA12, PA11, **PA10** (labeled USART1_RX), **PA9** (labeled USART1_TX), PA8, PC9, PC8, PC7, PC6, VDD.

RS485

- ❑ Long distance serial communication
- ❑ Differential signaling (2 wires for 1 bit stream)
- ❑ Distance: up to 1200m
- ❑ Max rate: up to 10 Mbps

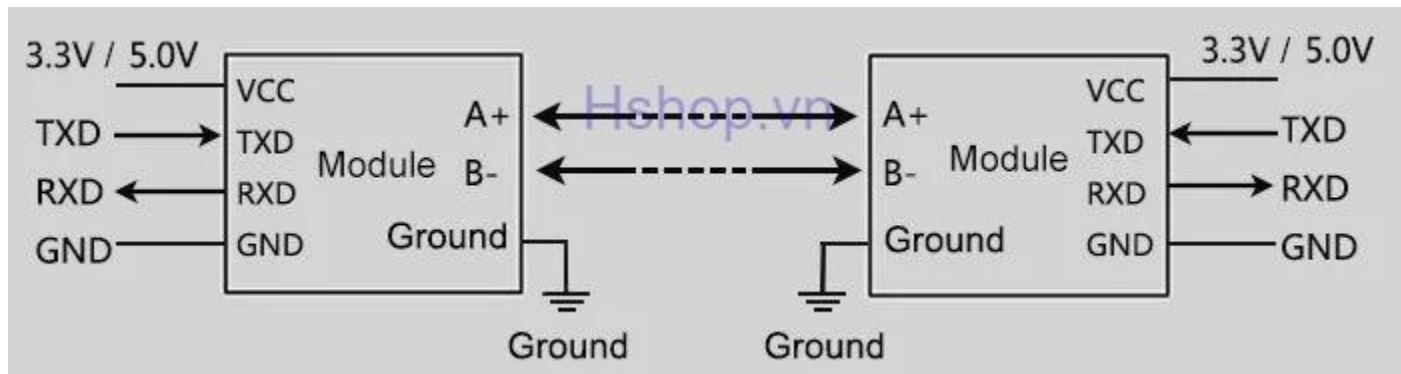
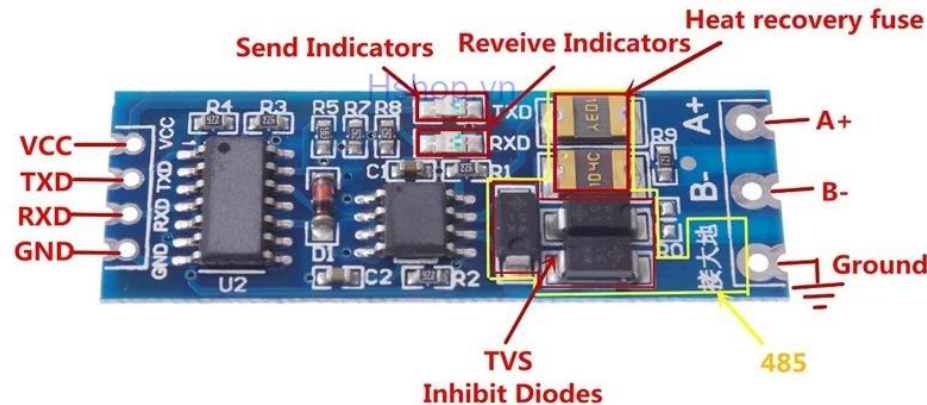


RS-232C comparison

Parameter	RS-232C	RS-422A	RS-485
Transmission mode	Simplex	Multi-point simplex	Multi-point multiplex
Max. connected devices	1 driver 1 receiver	1 driver 10 receivers	32 drivers 32 receivers
Max. transmission rate	20Kbps	10Mbps	10Mbps
Max. cable length	15m	1200m	1200m
Operation mode	Single-ended (unbalanced type)	Differential (balanced type)	Differential (balanced type)
Connection image			
Features	Short distance Full-duplex 1:1 connection	Long distance Full-duplex, half-duplex 1:N connection	Long distance Full-duplex, half-duplex N:N connection

Example: RS485 module

❑ Module UART - RS485



Connecting 2 nodes via RS485

➔ An appropriate protocol is required (e.g. Modbus)

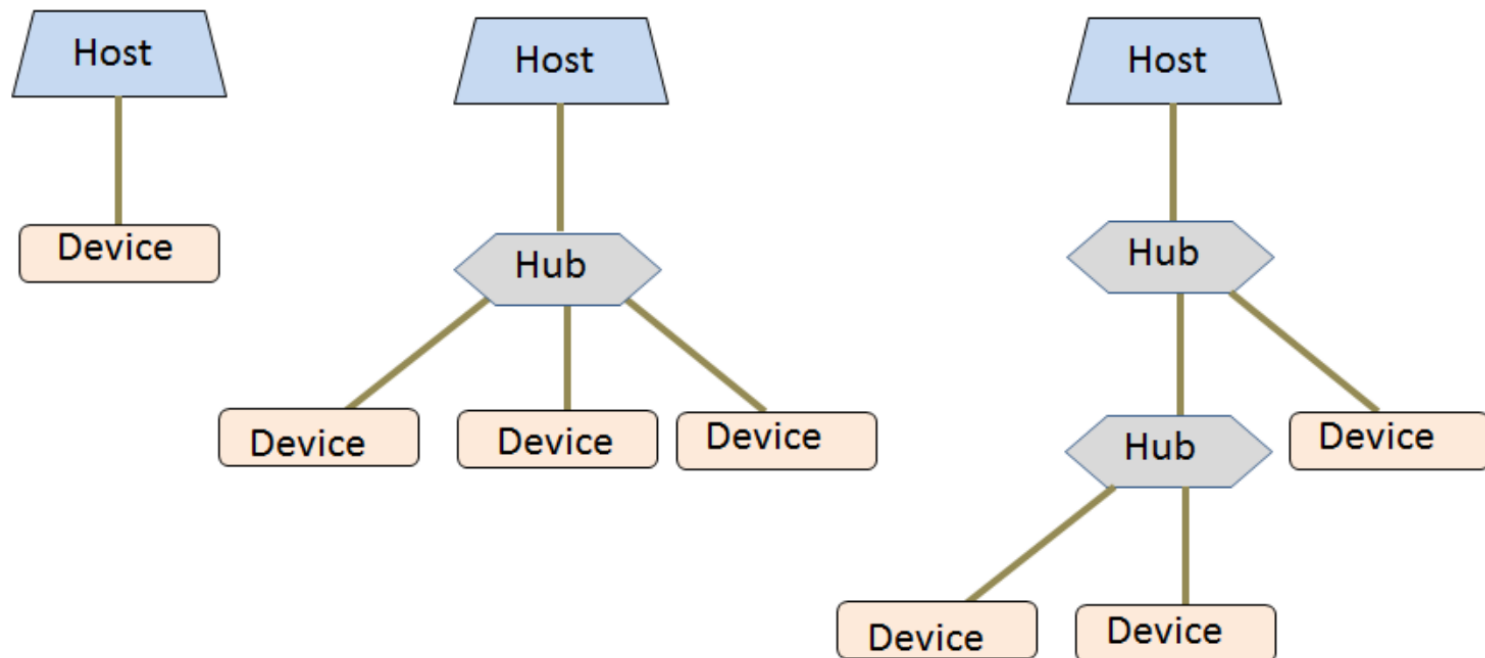
4.4. USB

- ❑ Universal Serial Bus, since 1994 (Compaq, DEC, IBM, Intel, Microsoft, NEC and Nortel.. now USB Implementers Forum, Inc.)
- ❑ Most popular interface (PC and handheld)
- ❑ (Ultra)High speed serial communication

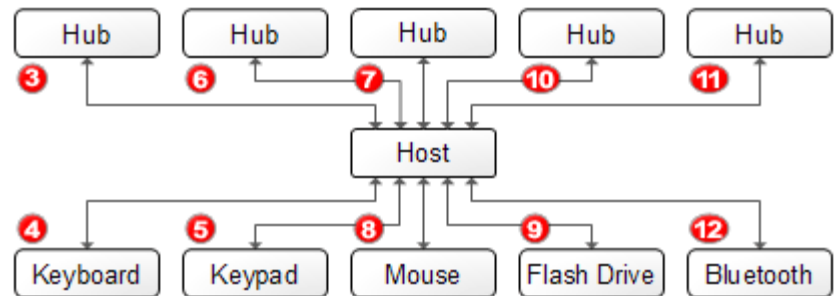
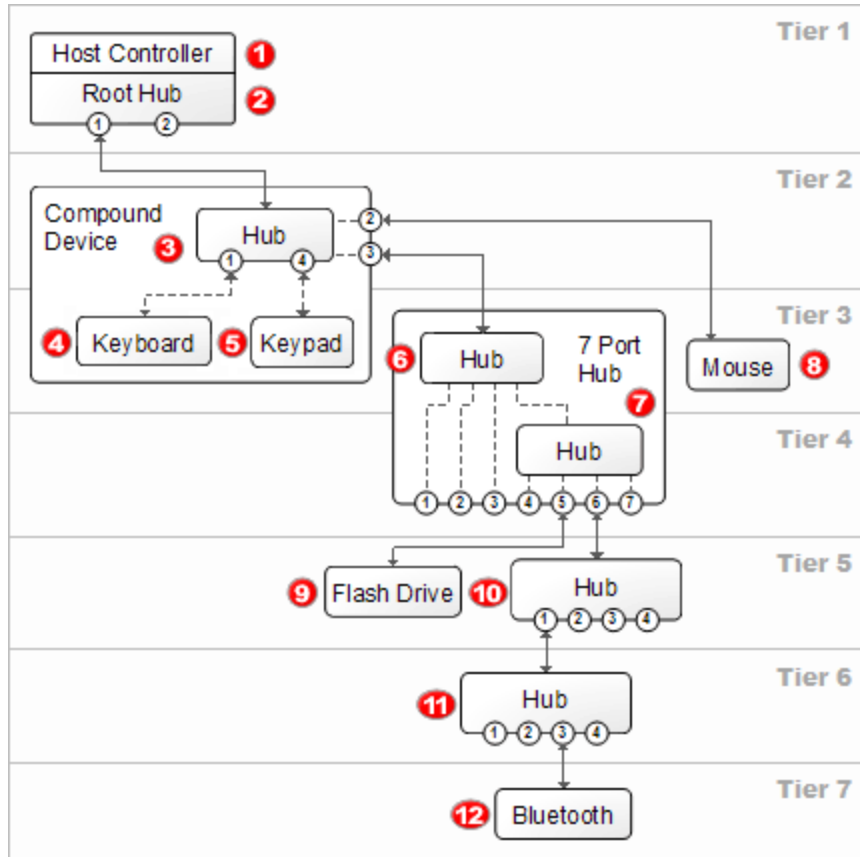
Connectors		USB 1.0 1996	USB 2.0 2001	USB 2.0 Revised	USB 3.0 2011	USB 3.1 2014	USB 3.2 2017	USB4 2019
Data rate		1.5 Mbit/s (Low Speed)	1.5 Mbit/s (Low Speed)		5 Gbit/s (SuperSpeed)	10 Gbit/s (SuperSpeed+)	20 Gbit/s (SuperSpeed+)	40 Gbit/s (SuperSpeed+ and Thunderbolt 3)
		12 Mbit/s (Full Speed)	12 Mbit/s (Full Speed) 480 Mbit/s (High Speed)					
Standard	A	Type A 			Type A  Type-A SuperSpeed		Deprecated	
	B	Type B 			Type B  Type-B SuperSpeed		Deprecated	
	C	N/A				Type C (enlarged) 		

Basic components

- ❑ USB host: the master device on USB bus, control every transactions on the bus.
- ❑ USB hub: special device to extend the bus.
- ❑ USB device: functional device, controlled by host.



Sample usb bus topology



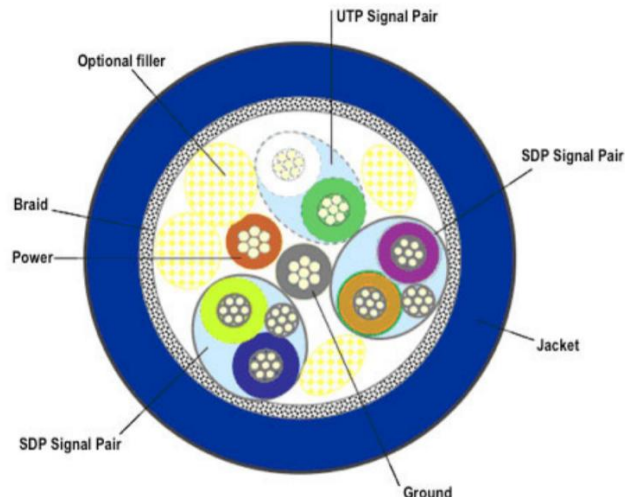
USB signals

USB 1.0/2.0

Pin	Name	Cable color	Description
1	VCC	Red	+5 VDC
2	D-	White	Data -
3	D+	Green	Data +
4	GND	Black	Ground

USB 3.x

Pin	Pin Name	Description
1	VBus	+5V Power
2	USB D-	USB 2.0 data
3	USB D+	
4	GND	Ground for power return
5	StdA_SSRX-	SuperSpeed receiver
6	StdA_SSRX+	SuperSpeed receiver
7	GND_DRAIN	Ground for signal return
8	StdA_SSTX-	SuperSpeed transmitter
9	StdA_SSTX+	SuperSpeed transmitter

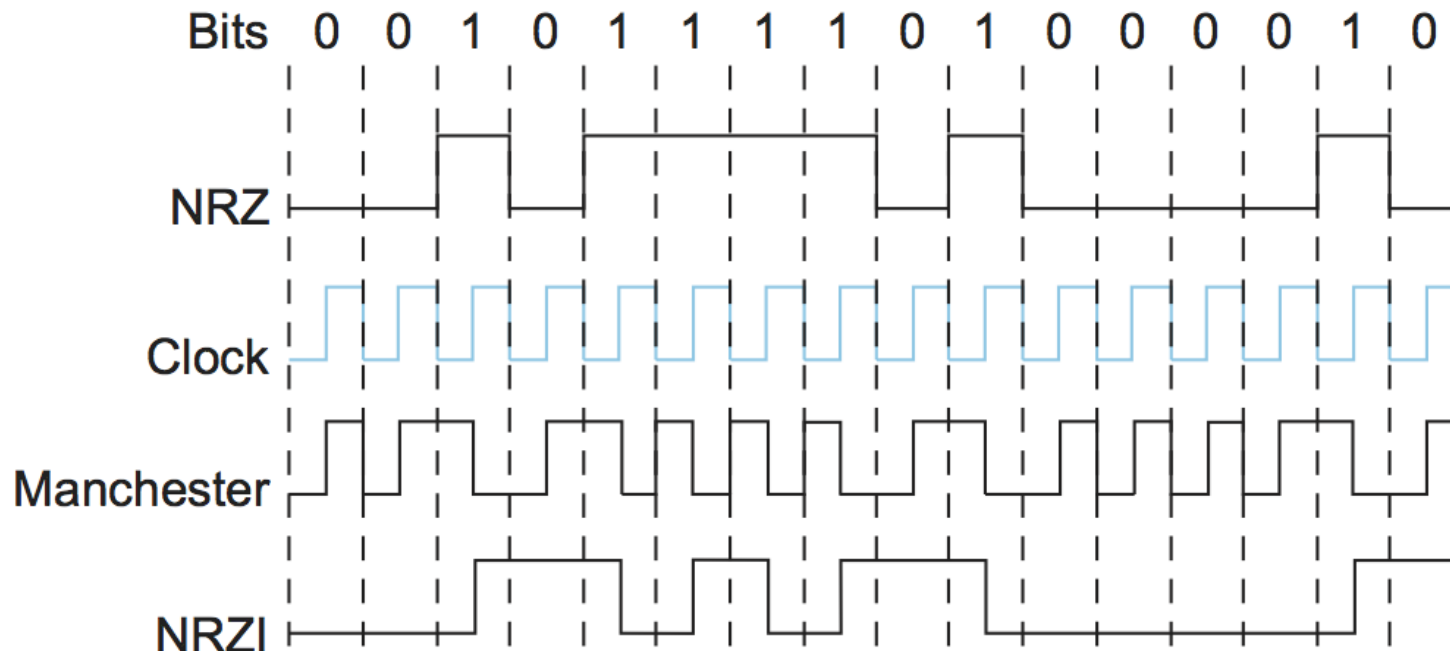


USB signals (2.0)

- ❑ No clock signal
- ❑ Data are transmitted by the pair D+/D-
 - | Logical high: 2.8 - 3.6V
 - | Logical low: 0.0 - 0.3 V
- ❑ Bus states
 - | Differential 0: D+ low and D- high
 - | Differential 1: D+ high and D- low

USB data encoding

- ❑ NRZI (Non-return to zero inverted)
- ❑ Used together with bit stuffing and SYNC mechanism
- ❑ Enable clock automatic recovery



USB bus transactions

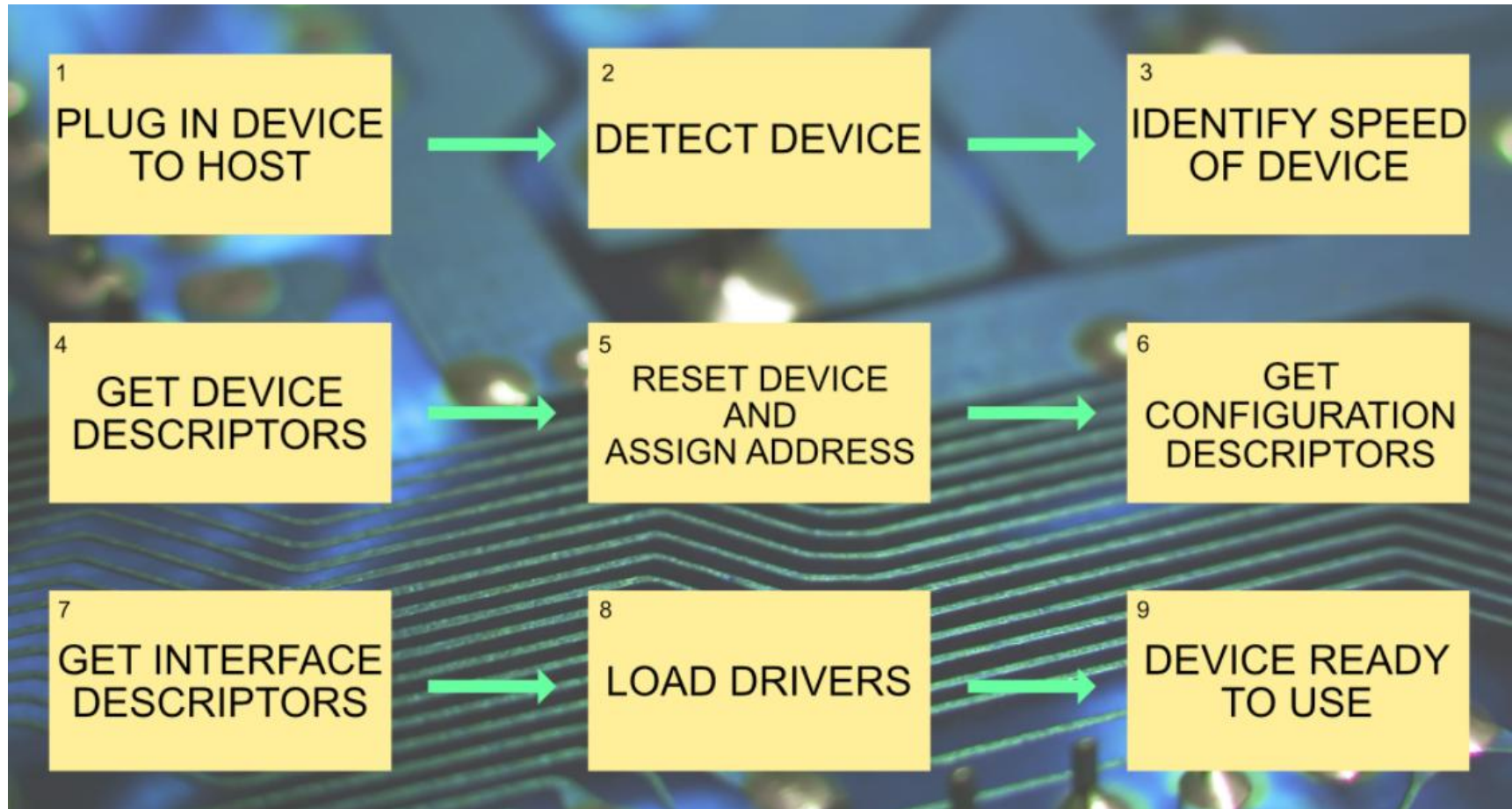
❑ USB Enumeration

- | The process from the moment the USB host detects a device connected to the bus, identifies the type of device, until it finishes initializing its operating configuration and loading the driver.
- | This process is performed once when the device is connected to the bus.
- | Each device has an address (from 1 to 127).
- | The default address is 0.

❑ Data transfer

- | USB host queries devices continuously.
- | Devices wants to transfer data will acknowledge.
 - Control transfer.
 - Interrupt transfer.
 - Isochronous transfer.
 - Bulk transfer.

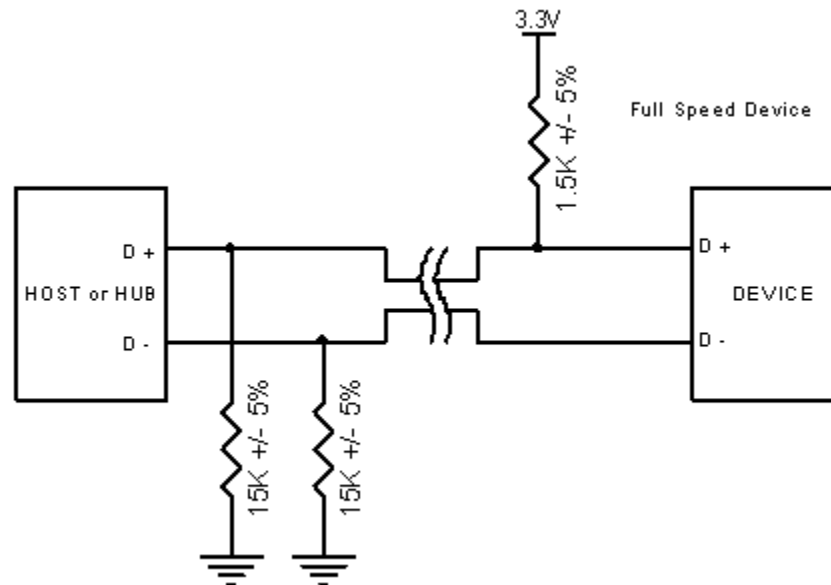
Bus enumeration process



<https://www.totalphase.com>

Bus enumeration: detect device

- When the device is plugged into the host/hub, the host detects an event indicating a new device.
- The host sends a reset signal to the new device → the new device is set to the default address = 00H. The device is allowed to draw a maximum current of 150 mA from the bus.



USB enumeration

- The host sends a **Get_Device_Descriptor** request to address 00H → the device returns the first 8 bytes of the **Device Descriptor** (to obtain the maximum packet size for Endpoint 0).
- The host sends a **Set_Address** request to assign a (unique) address to the device. The device receives and stores this new address.
- The host sends **Get_Device_Descriptor** again to retrieve the full **Device Descriptor**.
- The host decodes the **Device Descriptor** to obtain information about the device: device class, Product ID/Vendor ID, manufacturer name, etc.

USB Device Descriptor

Offset (decimal)	Field	Size (bytes)	Description
0	bLength	1	Descriptor size in bytes
1	bDescriptorType	1	The constant DEVICE (01h)
2	bcdUSB	2	USB specification release number (BCD)
4	bDeviceClass	1	Class code
5	bDeviceSubclass	1	Subclass code
6	bDeviceProtocol	1	Protocol Code
7	bMaxPacketSize0	1	Maximum packet size for Endpoint 0
8	idVendor	2	Vendor ID
10	idProduct	2	Product ID
12	bcdDevice	2	Device release number (BCD)
14	iManufacturer	1	Index of string descriptor for the manufacturer
15	iProduct	1	Index of string descriptor for the product
16	iSerialNumber	1	Index of string descriptor containing the serial number
17	bNumConfigurations	1	Number of possible configurations

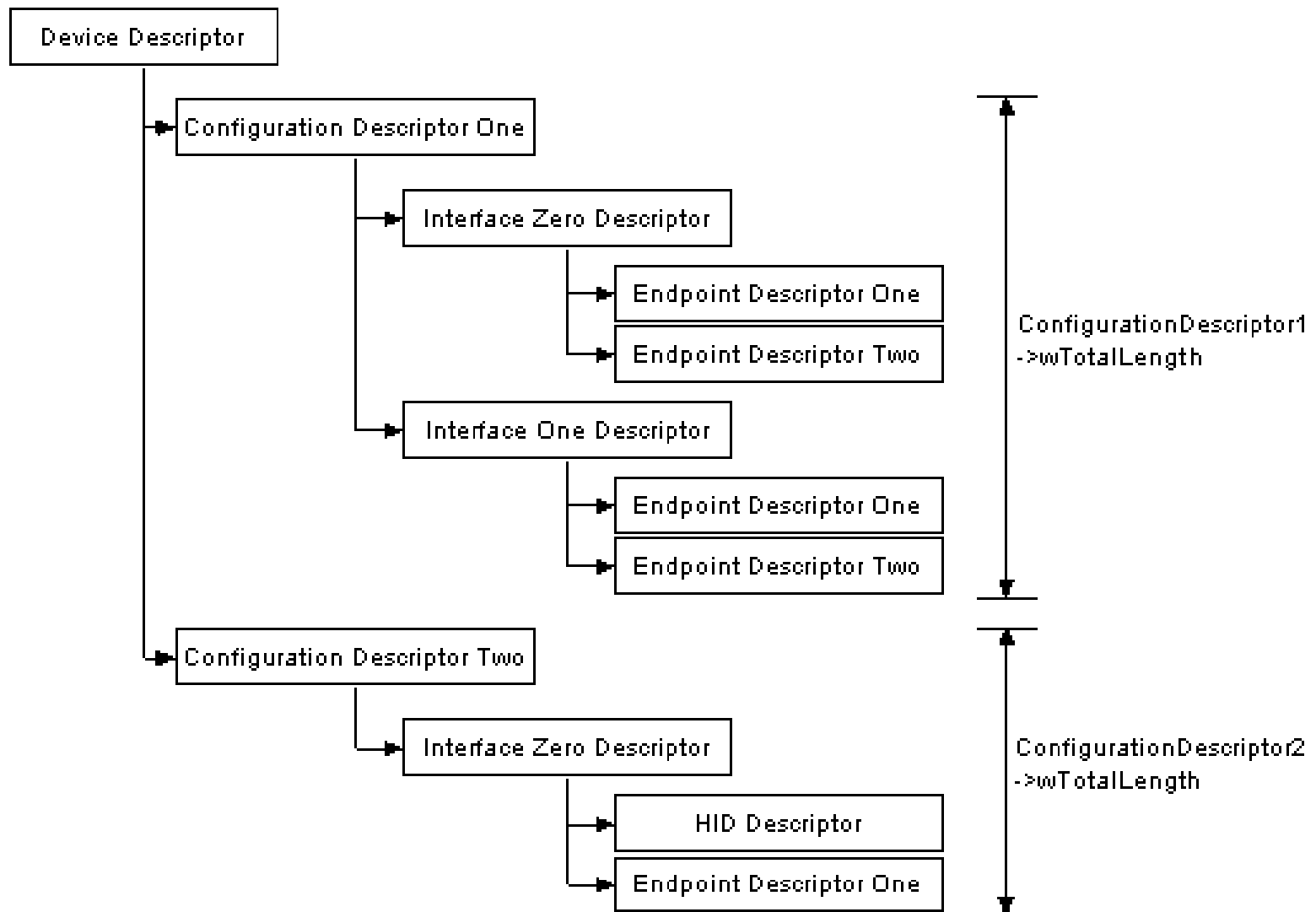
USB enumeration

- Host sends **Get_Configuration_Descriptor** and **Get_Interface_Descriptor** to get additional data for initializing device driver.

Offset (decimal)	Field	Size (bytes)	Description
0	bLength	1	Descriptor size in bytes
1	bDescriptorType	1	The constant Configuration (02h)
2	wTotalLength	2	The number of bytes in the configuration descriptor and all of its subordinate descriptors
4	bNumInterfaces	1	Number of interfaces in the configuration
5	bConfigurationValue	1	Identifier for Set_Configuration and Get_Configuration requests
6	iConfiguration	1	Index of string descriptor for the configuration
7	bmAttributes	1	Self/bus power and remote wakeup settings
8	bMaxPower	1	Bus power required, expressed as (maximum milliamperes/2)

Offset (decimal)	Field	Size (bytes)	Description
0	bLength	1	Descriptor size in bytes
1	bDescriptorType	1	The constant Interface (04h)
2	bInterfaceNumber	1	Number identifying this interface
3	bAlternateSetting	1	Value used to select an alternate setting
4	bNumEndpoints	1	Number of endpoints supported, not counting Endpoint 0
5	bInterfaceClass	1	Class code
6	bInterfaceSubclass	1	Subclass code
7	bInterfaceProtocol	1	Protocol code
8	iInterface	1	Index of string descriptor for the interface

USB descriptor



USB Enumeration

- ❑ Finally, the host loads the device driver corresponding to the device.
- ❑ The operating system (or the host program) determines the driver based on the Product ID/Vendor ID pair in order to load the appropriate INF file.
- ❑ The host sends a Set_Configuration request to set the device's operating configuration.
- ❑ The device enters the Configured state and begins exchanging data (under the control of the host).

USB device classes

[About USB-IF](#)[Developers](#)[Compliance](#)[Logo License Request](#)[Pressroom](#)

Last Update: Septmeber 22, 2023

Base Class	Descriptor Usage	Description
00h	Device	Use class information in the Interface Descriptors
01h	Interface	Audio
02h	Both	Communications and CDC Control
03h	Interface	HID (Human Interface Device)
05h	Interface	Physical
06h	Interface	Image
07h	Interface	Printer
08h	Interface	Mass Storage
09h	Device	Hub
0Ah	Interface	CDC-Data
0Bh	Interface	Smart Card
0Dh	Interface	Content Security
0Eh	Interface	Video

[Defined Class Codes | USB-IF](#)

USB Human Interface Devices (HID)

- ❑ Use for devices that enables human-computer interaction: USB mouse, USB keyboards, USB joysticks, knobs, switches, buttons, and sliders...
- ❑ Defines in USB HID specification.
- ❑ Class code: 0x03
- ❑ Protocol value: specify types of device

Protocol value	Device type
1	USB keyboard
2	USB mouse

USB keyboard report format

- Up to 8 bytes per report

Offset	Size	Description
0	Byte	Modifier keys status.
1	Byte	Reserved field.
2	Byte	Keypress #1.
3	Byte	Keypress #2.
4	Byte	Keypress #3.
5	Byte	Keypress #4.
6	Byte	Keypress #5.
7	Byte	Keypress #6.

USB keyboard report data format

Bit	Bit Length	Description
0	1	Left Ctrl.
1	1	Left Shift.
2	1	Left Alt.
3	1	Left GUI (Windows/Super key.)
4	1	Right Ctrl.
5	1	Right Shift.
6	1	Right Alt.
7	1	Right GUI (Windows/Super key.)

Bitmap of Modifier keys status

- Eg. press and hold key A, then press key B

00 00 04 05 00 00 00 00

USB mouse report format

- ❑ Up to 4 bytes per report
 - | Offset 0: button status
 - | Offset 1: X movement
 - | Offset 2: Y movement
 - | Offset 3: wheel movement

Bit	Bit Length	Description
0	1	When set to 1, indicates the left mouse button is being clicked.
1	1	When set to 1, indicates the right mouse button is being clicked.
2	1	When set to 1, indicates the middle mouse button is being clicked.
3	5	These bits are reserved for device-specific features.

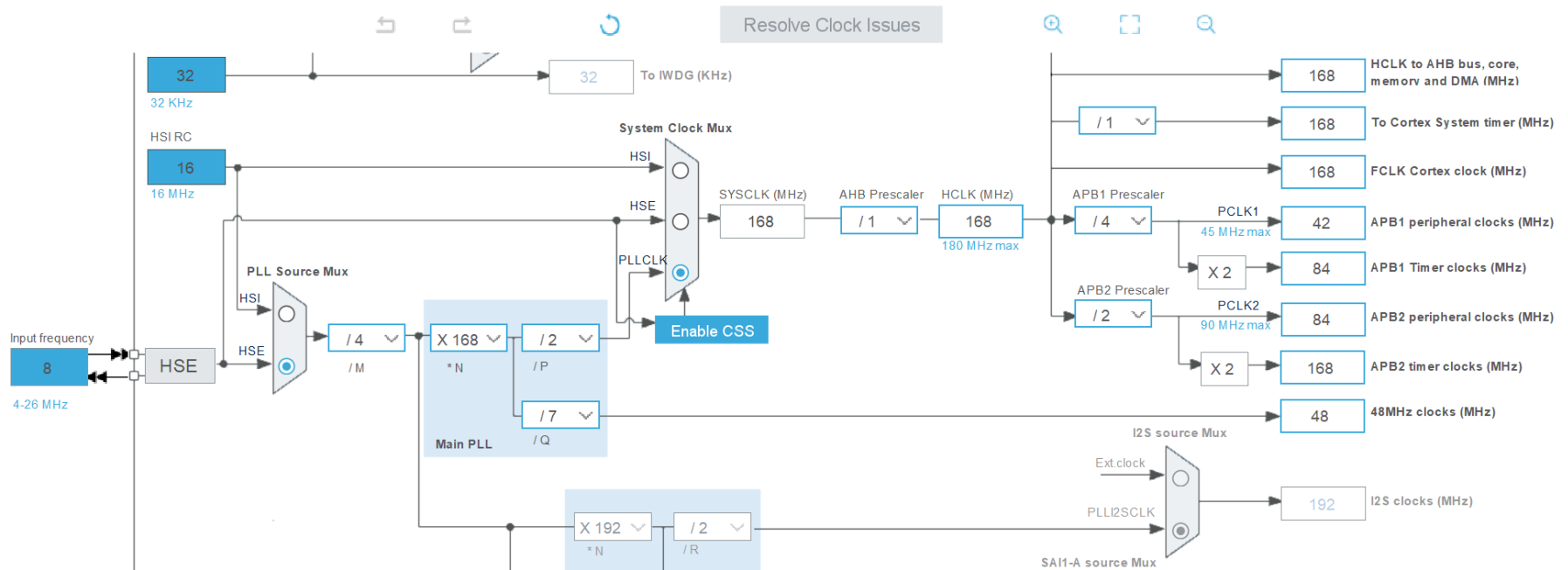
USB mouse button status bitmap

Exercise: USB mouse

- ❑ Develop firmware for the STM32F429 to operate as an HID device using the USB mouse profile. Also each time the PA0 button is pressed, a left-mouse-click event is sent to the PC.

Exercise: USB mouse

- ❑ Create new project, configure RCC and HCLK = 168 MHz (to enable 48 MHz clock for USB).



Exercise: USB mouse

❑ Further configuration for GPIO and USB

- | PA0: GPIO External interrupt Rising/Falling edges, enable GPIO_EXTI0 in NVIC.
- | USB_OTG_HS: Internal FS Phy **Device_Only**

The screenshot displays the STM32CubeMX Pinout & Configuration window. The 'Pinout & Configuration' tab is active, showing the 'USB_OTG_HS Mode and Configuration' settings. The 'Mode' section has 'External Phy' set to 'Disable' and 'Internal FS Phy' set to 'Device_Only' (highlighted with a red underline). The 'Configuration' section shows 'Activate_SOF' as unchecked and 'Activate_VBUS' as checked. The 'Configuration' section also includes a 'Reset Configuration' button and tabs for 'NVIC Settings', 'GPIO Settings', 'Parameter Settings', and 'User Constants'. The 'Parameter Settings' tab is selected, showing 'Speed' set to 'Device Full Speed 12M...' and 'Enable internal I' set to 'Disabled'. The 'Pinout view' on the right shows the pin configuration for the USB_OTG_HS module, with pins PB15, PB14, and PB13 highlighted in green and labeled as USB_OTG_HS_DP, USB_OTG_HS_DM, and USB_OTG_HS_VBUS respectively. The 'System' view on the right shows the pin configuration for the system, with pins PE12, PE13, PE14, PE15, PB10, PB11, VCAP, and VDD highlighted in yellow.

Pinout & Configuration | Clock Configuration | Project Manager

Software Packs | Pinout

USB_OTG_HS Mode and Configuration

Mode

External Phy: Disable

Internal FS Phy: Device_Only

☐ Activate_SOF

☒ Activate_VBUS

Configuration

Reset Configuration

NVIC Settings | GPIO Settings

Parameter Settings | User Constants

Configure the below parameters :

Search (Ctrl+F)

Speed: Device Full Speed 12M...

Enable internal I: Disabled

Pinout view | System

PD13, PD12, PD11, PD10, PD9, PD8, PB15, PB14, PB13, PB12

USB_OTG_HS_DP, USB_OTG_HS_DM, USB_OTG_HS_VBUS

PE12, PE13, PE14, PE15, PB10, PB11, VCAP, VDD

Exercise: USB mouse

- ❑ Configure Middleware and Software component
 - | Select USB_DEVICE
 - | Set class for HS IP as **Human Interface Device (HID)**

The screenshot displays the STM32CubeMX Pinout & Configuration window. The 'Pinout & Configuration' tab is active, showing the 'USB_DEVICE Mode and Configuration' section. The 'Mode' section has 'Class For HS IP' set to 'Human Interface Devic...' and 'Class For FS IP' set to 'Disable'. The 'Configuration' section includes a 'Reset Configuration' button and a 'User Constants' section with 'Parameter Settings' and 'Device Descriptor' tabs. The 'Device Descriptor' tab is selected, showing parameters like VID (Vendor ID..1155), LANGID_STRI.. English(United States), and MANUFACTU... STMicroelectronics. The 'Device Descriptor HS' section is also visible. On the left, the 'Categories' list shows 'USB_DEVICE' selected. On the right, the 'Pinout' section shows a pin list with PD13, PD12, PD11, PD10, PD9, PD8, PB15, PB14, PB13, and PB12.

Pinout & Configuration | Clock Configuration | Project Manager

Software Packs | Pinout

USB_DEVICE Mode and Configuration

Mode

Class For HS IP: Human Interface Devic...
Class For FS IP: Disable

Configuration

Reset Configuration

User Constants

Parameter Settings | Device Descriptor

Configure the below parameters :

Search (Ctrl+F)

Device Descriptor

VID (Vendor ID..1155)
LANGID_STRI.. English(United States)
MANUFACTU... STMicroelectronics

Device Descriptor HS

Pinout

PD13
PD12
PD11
PD10
PD9
PD8
PB15
PB14
PB13
PB12

PE12 PE13 PE14 PE15 PB10 PB11 VCAP... VDD

Exercise: USB mouse

- ❑ Declare the mouseHID struct in main.h to encapsulate the 4 byte USB Mouse report.
 - | button: 0x01 for left button, 0x02 for right button.
 - | mouse_x, mouse_y: movement along x and y axis.
 - | wheel: wheel scroll.

```
37- /* Exported types -----  
38  /* USER CODE BEGIN ET */  
39- typedef struct  
40  {  
41      uint8_t button;  
42      int8_t mouse_x;  
43      int8_t mouse_y;  
44      int8_t wheel;  
45  } mouseHID;  
46  /* USER CODE END ET */
```

Exercise: USB mouse

- ❑ Declare an extern var in main.c (this is defined in another class in USB driver)

```
58- /* Private user code -----  
59  /* USER CODE BEGIN 0 */  
60  #include "usbd_hid.h"  
61  
62  extern USB_D_HandleTypeDef hUsbDeviceHS;  
63  
64  mouseHID mousehid = {0,0,0,0};  
65  /* USER CODE END 0 */
```

Exercise: USB mouse

❑ Add simulated mouse movement code in main loop

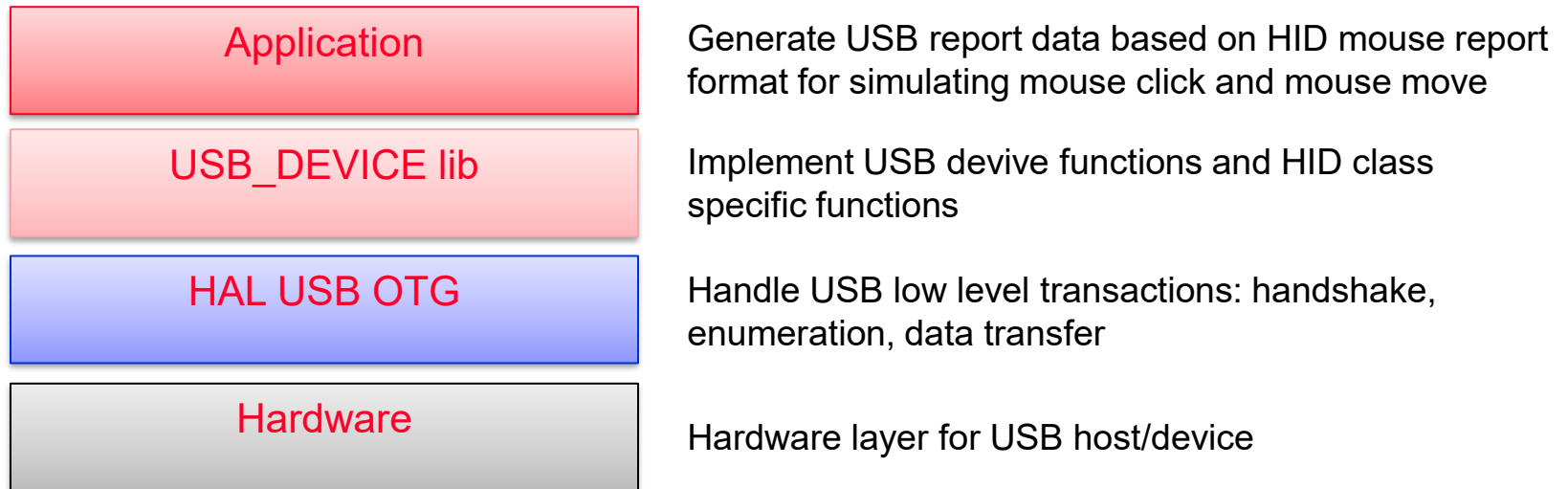
```
94  /* Initialize all configured peripherals */
95  MX_GPIO_Init();
96  MX_USART1_UART_Init();
97  MX_USB_DEVICE_Init();
98  /* USER CODE BEGIN 2 */
99
100 /* USER CODE END 2 */
101
102 /* Infinite loop */
103 /* USER CODE BEGIN WHILE */
104 while (1)
105 {
106     int button_pressed = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
107     if (button_pressed)
108     {
109         mousehid.mouse_x = 1;
110         mousehid.mouse_y = 1;
111         USBD_HID_SendReport(&hUsbDeviceHS, (uint8_t *)&mousehid, sizeof (mousehid));
112     }
113
114     HAL_Delay(100);
115     /* USER CODE END WHILE */
116
117     /* USER CODE BEGIN 3 */
118 }
119 /* USER CODE END 3 */
```

Exercise: USB mouse

- ❑ Run code on STM32F429 board.
- ❑ Plug the micro USB cable to USB USER port and connect to PC.
- ❑ Observe on PC: mouse move when the User button is pressed.

Exercise: USB mouse

❑ Project structure



Exercise

- ❑ Handle the User button interrupt to send Mouse up/down event to PC when the button is pressed/released.